



TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO CUỐI KỲ NHẬP MÔN TRÍ TUỆ NHÂN TẠO

GVPT: ThS. Nguyễn Quốc Khoa

Ca:



1

Họ và tên SV: Huỳnh Ngọc Huy

MSSV: 24200087

Thành viên nhóm: Huỳnh Ngọc Huy - 24200087
Choi Won Seok - 24200032

KHTN

ĐHQG-HCM

Mục lục

1	Giới Thiệu	6
1.1	Đặt Vấn Đề	6
1.2	Mục Tiêu Đề Án	6
1.3	Phương Pháp Tiếp Cận	6
2	Mô Tả Bộ Dữ Liệu	7
2.1	Tổng Quan về GTSRB	7
2.2	Cấu Trúc Thư Mục	7
2.3	Đặc Điểm Dữ Liệu	7
2.4	Phân Tích Mất Cân Bằng Dữ Liệu	7
2.5	Biểu Đồ Phân Phối Lớp	8
3	Phương Pháp Thực Hiện	8
3.1	Cấu Hình Toàn Cục	8
3.2	Bước 1: Đọc Dữ Liệu – <code>load_data()</code>	9
3.3	Bước 2: Phân Tích và Xử Lý Mất Cân Bằng – Class Weights (9 điểm) .	10
3.4	Bước 3: Chuẩn Hóa Dữ Liệu và Data Augmentation	11
3.5	Bước 4: Mã Hóa Nhãn (One-Hot Encoding)	12
3.6	Bước 5: Chia Dữ Liệu	12
3.7	Bước 6: Xây Dựng Mô Hình CNN	12
3.8	Bước 7: Biên Dịch và Huấn Luyện	15
3.9	Bước 8: Đánh Giá và Lưu Mô Hình	16
4	So Sánh Các Thuật Toán	17
4.1	Tổng Quan Các Mô Hình Được So Sánh	17
4.2	Kiến Trúc Chi Tiết Các Mô Hình	18
4.3	Phân Tích Chuyên Sâu Các Kiến Trúc	20
5	Quy Trình Hoạt Động	22
6	Hướng Dẫn Chạy Chương Trình	23
6.1	Yêu Cầu Cài Đặt	23
6.2	Các Lệnh Chạy	23
6.3	Kết Quả Mong Đợi	24
7	Kết Quả Thực Nghiệm	24
7.1	Môi Trường Thực Nghiệm	24
7.2	Kết Quả <code>trafficsign.py</code> – Baseline CNN + Class Weights (8 điểm) .	24
7.3	Kết Quả <code>trafficsign_new.py</code> – So Sánh 4 Mô Hình (8–9–10 điểm) . .	25
7.4	Bảng So Sánh 4 Mô Hình	33
7.5	Phân Tích Chuyên Sâu	33
7.6	Classification Report Chi Tiết	35
7.7	Phân Tích Ma Trận Nhầm Lẫn	44
7.8	Kết Quả Inference – Kiểm Tra Mô Hình Trên Ảnh Thực Tế	44

8	Kết Luận	54
8.1	Tóm Tắt Kết Quả	54
8.2	Bài Học Kinh Nghiệm	54
8.3	Hướng Phát Triển	55

Danh sách hình vẽ

1	Biểu đồ phân phối số lượng ảnh trên 43 lớp của GTSRB. Đường đồ thể hiện mức trung bình 619 ảnh/lớp. Các lớp 0, 19, 37–42 có ít hơn 200 ảnh, trong khi lớp 1 có đến 1500 ảnh.	8
2	Sơ đồ kiến trúc mô hình CNN hiển thị qua <code>model.summary()</code> và <code>tf.keras.utils.plot_model</code> . Các tầng Conv2D, MaxPooling2D, Dropout, Flatten, Dense được hiển thị với kích thước đầu ra và số tham số.	13
3	Quy trình hoạt động đầy đủ của <code>trafficsign_new.py</code>	22
4	Biểu đồ đường cong huấn luyện của Model A (Baseline CNN + Class Weights). Bên trái: Accuracy (train + validation) qua 10 epoch. Bên phải: Loss (train + validation) qua 10 epoch.	26
5	Biểu đồ đường cong huấn luyện của Model B (CNN + L2 Regularization + Class Weights). L2 regularization với $\lambda = 0.001$ có tác động nhẹ đến quá trình hội tụ – test accuracy (96.53%) gần bằng Model A (97.24%).	28
6	Biểu đồ đường cong huấn luyện của Model C – Phần 1: Toàn bộ 4 mô hình trên cùng một biểu đồ. Model C (Deeper CNN + BN) cho thấy đường cong hội tụ nhanh và ổn định nhất.	30
7	Biểu đồ đường cong huấn luyện của Model C – Phần 2: Biểu đồ riêng của Model C. Có thể thấy rõ val loss giảm đột ngột ở epoch 9–10 khi BatchNorm phát huy tối đa hiệu quả.	31
8	Biểu đồ đường cong huấn luyện của Model D (MobileNetV2 Transfer Learning). Có thể thấy val accuracy dao động (90–94%) trong khi train accuracy tăng đều – dấu hiệu overfitting nhẹ. Tuy nhiên MobileNetV2 hội tụ nhanh nhất ở epoch 1 (38.14%) nhờ trọng số pre-trained.	32
9	Kết quả cuối cùng khi chạy <code>trafficsign_new.py</code> : Bảng so sánh 4 mô hình với Test Accuracy. Model C đạt kết quả cao nhất 98.55%.	33
10	Classification Report của Model A (Baseline CNN + Class Weights) – Test Accuracy: 97.24%. Kết quả đầu ra từ <code>sklearn.metrics.classification_report</code> với precision, recall, F1-score cho từng lớp.	36
11	Classification Report của Model B (CNN + L2 Reg + Class Weights) – Test Accuracy: 96.53%. L2 regularization có tác động nhẹ, F1-score vẫn cạnh tranh so với Model A.	38
12	Classification Report của Model C (Deeper CNN + BatchNorm) – Test Accuracy: 98.55%. Đây là mô hình tốt nhất với Macro Avg F1-score = 0.988.	40
13	Classification Report của Model D (MobileNetV2 Transfer Learning) – Test Accuracy: 95.42%. Mặc dù có accuracy thấp nhất, MobileNetV2 vẫn đạt F1-score trên 0.95 ở nhiều lớp.	43
14	Kết quả dự đoán 4 ảnh kiểm tra bằng mô hình tốt nhất (Model C). Mỗi ảnh hiển thị top-3 dự đoán kèm confidence score và thanh trực quan.	44
15	Dự đoán ảnh kiểm tra số 1 – Biển báo giới hạn tốc độ. Mô hình dự đoán chính xác với độ tin cậy cao.	45
16	Dự đoán ảnh kiểm tra số 2 – Biển báo đường ưu tiên. Mô hình dự đoán đúng với confidence gần như tuyệt đối.	46
17	Dự đoán ảnh kiểm tra số 3 – Biển báo cấm xe tải. Mô hình phân biệt chính xác giữa các loại biển báo cấm.	47

18	Dự đoán ảnh kiểm tra số 4 – Biển báo nguy hiểm. Mô hình nhận diện đúng loại biển báo nguy hiểm.	48
19	Kết quả dự đoán 4 ảnh kiểm tra bằng mô hình cơ bản (<code>trafficsign.py</code>). So sánh với mô hình tốt nhất, có thể thấy sự khác biệt về confidence score.	48
20	Dự đoán ảnh số 1 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.	49
21	Dự đoán ảnh số 2 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.	50
22	Dự đoán ảnh số 3 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.	51
23	Dự đoán ảnh số 4 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.	52
24	Kết quả inference của Model C (Deeper CNN + BatchNorm) trên toàn bộ 12,630 ảnh GTSRB Test Set. Kết quả: 11,933/12,630 đúng (94.48%).	52
25	Kết quả inference của Model A (Baseline CNN) trên toàn bộ 12,630 ảnh GTSRB Test Set. Kết quả: 12,146/12,630 đúng (96.17%).	53

Danh sách bảng

1	Phân tích sự mất cân bằng dữ liệu GTSRB.	7
2	Siêu tham số toàn cục.	8
3	Kiến trúc Baseline CNN. Đầu vào (30, 30, 3).	13
4	Diễn biến huấn luyện <code>trafficsign.py</code> (Baseline CNN + Class Weights, 30×30).	25
5	Diễn biến huấn luyện Model A (Baseline CNN + Class Weights, 30×30 , data augmentation).	25
6	Diễn biến huấn luyện Model B (CNN + L2 Regularization + Class Weights).	27
7	Diễn biến huấn luyện Model C (Deeper CNN 3 Blocks + BatchNorm) – mô hình tốt nhất.	29
8	Diễn biến huấn luyện Model D (Transfer Learning với MobileNetV2). .	31
9	So sánh hiệu suất 4 thuật toán.	33
10	Classification Report cho Model A (Baseline CNN + Class Weights) – Test Accuracy: 97.24%.	35
11	Classification Report cho Model B (CNN + L2 Reg + Class Weights) – Test Accuracy: 96.53%.	37
12	Classification Report cho Model C (Deeper CNN + BatchNorm) – Test Accuracy: 98.55%.	39
13	Classification Report cho Model D (MobileNetV2 Transfer Learning) – Test Accuracy: 95.42%.	42
14	Phân tích các lớp khó nhận diện nhất cho từng mô hình.	44
15	Kết quả inference trên 4 thư mục ảnh tùy chỉnh. Model C nhỉnh hơn Model A 1 ảnh (114 vs 113 trên tổng 120 ảnh).	53
16	Kết quả inference trên toàn bộ GTSRB Test Set (12,630 ảnh). Model A đạt độ chính xác cao hơn Model C trên tập test chuẩn.	53

Giới Thiệu

Đặt Vấn Đề

Nhận diện biển báo giao thông là một thành phần quan trọng trong hệ thống xe tự lái và hệ thống hỗ trợ người lái tiên tiến (ADAS). Mục tiêu là tự động phân loại ảnh biển báo giao thông vào đúng danh mục (ví dụ: giới hạn tốc độ, dừng, nhường đường) từ hình ảnh camera thu được từ xe.

Bài toán này gặp nhiều thách thức do sự thay đổi về điều kiện ánh sáng, vật che khuất, mờ chuyển động và thời tiết khác nhau. Các phương pháp truyền thống như trích xuất đặc trưng thủ công (HOG, SIFT) kết hợp SVM gặp khó khăn khi xử lý 43 loại biển báo khác nhau với chất lượng ảnh không đồng nhất.

Mục Tiêu Đề Án

Đề án này xây dựng một hệ thống nhận diện biển báo giao thông sử dụng Convolutional Neural Network (CNN) với TensorFlow/Keras trên bộ dữ liệu German Traffic Sign Recognition Benchmark (GTSRB). Mô hình phải xác định chính xác mỗi ảnh đầu vào thuộc 1 trong 43 loại biển báo.

Các mục tiêu cụ thể:

1. Xây dựng pipeline hoàn chỉnh từ đọc dữ liệu đến huấn luyện và đánh giá mô hình CNN.
2. Xử lý vấn đề mất cân bằng dữ liệu giữa các lớp bằng class weights.
3. Thử nghiệm và so sánh nhiều thuật toán/kiến trúc khác nhau.
4. Đánh giá chi tiết bằng classification report (precision, recall, F1-score).
5. Xây dựng công cụ inference cho ảnh đơn và thư mục ảnh.

Phương Pháp Tiếp Cận

Chúng tôi áp dụng phương pháp học sâu có giám sát theo quy trình 8 bước:

Bước 1: Đọc và tiền xử lý ảnh biển báo giao thông từ bộ dữ liệu GTSRB.

Bước 2: Phân tích phân phối lớp và tính class weights để xử lý mất cân bằng.

Bước 3: Chuẩn hóa dữ liệu về khoảng $[0, 1]$ và áp dụng data augmentation trong quá trình huấn luyện.

Bước 4: Mã hóa nhãn bằng one-hot encoding cho bài toán phân loại đa lớp.

Bước 5: Chia dữ liệu thành tập huấn luyện (80%) và tập kiểm tra (20%).

Bước 6: Xây dựng kiến trúc CNN và biên dịch mô hình với Adam optimizer.

Bước 7: Huấn luyện mô hình trong 10 epoch và đánh giá trên tập kiểm tra.

Bước 8: So sánh kết quả giữa các thuật toán khác nhau.

Mô Tả Bộ Dữ Liệu

Tổng Quan về GTSRB

German Traffic Sign Recognition Benchmark (GTSRB) chứa hơn 50.000 ảnh biển báo giao thông được chia thành 43 lớp riêng biệt. Mỗi lớp tương ứng với một loại biển báo giao thông của Đức. Đây là bộ dữ liệu chuẩn được sử dụng rộng rãi trong nghiên cứu nhận diện biển báo.

Cấu Trúc Thư Mục

Bộ dữ liệu được tổ chức dưới dạng cây thư mục:

```
gtsrb/
+-- 0/          # Giới hạn tốc độ 20 km/h
+-- 1/          # Giới hạn tốc độ 30 km/h
+-- 2/          # Giới hạn tốc độ 50 km/h
...
+-- 42/         # Hạn chế vượt (xe tải)
```

Mỗi thư mục con chứa các tập tin ảnh PNG/PPM với kích thước khác nhau, tất cả thuộc cùng một loại biển báo. Tên thư mục (0–42) đóng vai trò là nhãn nguyên cho danh mục đó.

Đặc Điểm Dữ Liệu

- **Số lớp:** 43
- **Định dạng ảnh:** PPM / PNG (màu RGB)
- **Kích thước ảnh thay đổi:** Cần thay đổi kích thước trước khi đưa vào mạng. Cả hai chương trình `trafficsign.py` và `trafficsign_new.py` đều resize ảnh về 30×30 .
- **Kích thước đầu vào:** $30 \times 30 \times 3$ cho tất cả mô hình (riêng MobileNetV2 resize lên 96×96 bên trong mô hình).

Phân Tích Mất Cân Bằng Dữ Liệu

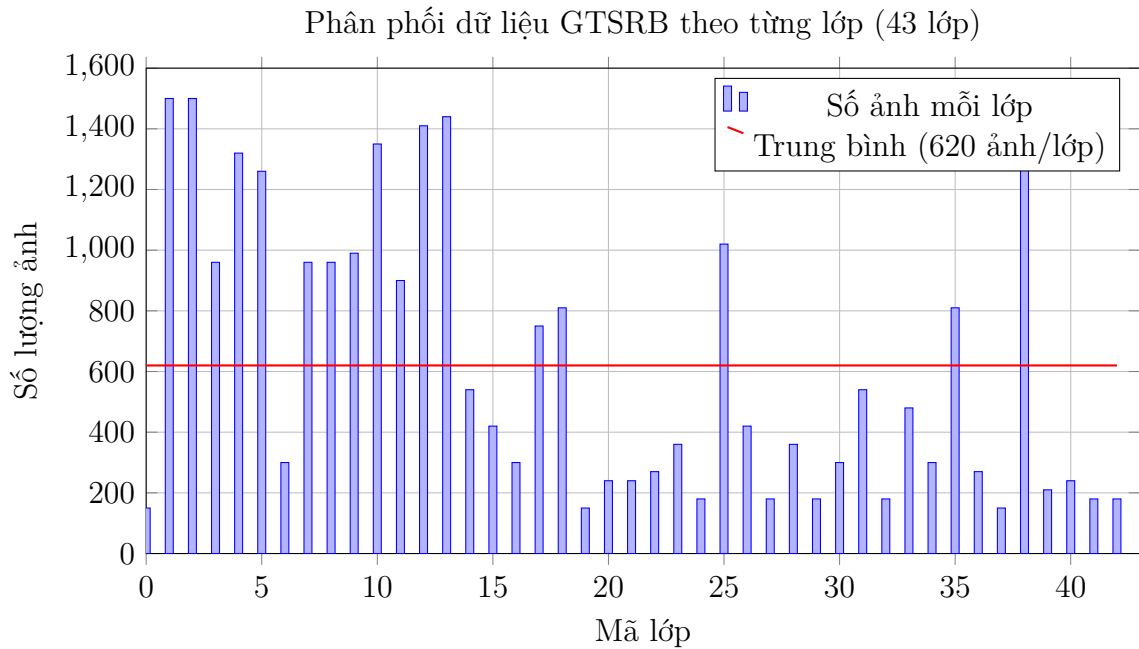
Bộ dữ liệu GTSRB có sự mất cân bằng đáng kể giữa các lớp:

Chỉ số	Giá trị
Lớp ít ảnh nhất	Lớp 0 (150 ảnh)
Lớp nhiều ảnh nhất	Lớp 1 (1500 ảnh)
Tổng số ảnh	26.640
Tỷ lệ mất cân bằng (max/min)	10:1

Bảng 1: Phân tích sự mất cân bằng dữ liệu GTSRB.

Sự chênh lệch lớn (lớp 1 có gấp 10 lần lớp 0) có thể ảnh hưởng tiêu cực đến hiệu suất mô hình: mô hình sẽ thiên vị dự đoán các lớp đa số và bỏ qua các lớp thiểu số. Đây là vấn đề quan trọng cần được xử lý để đạt điểm cao (mức 9 điểm).

Biểu Đồ Phân Phối Lớp



Hình 1: Biểu đồ phân phối số lượng ảnh trên 43 lớp của GTSRB. Đường đỏ thể hiện mức trung bình 619 ảnh/lớp. Các lớp 0, 19, 37–42 có ít hơn 200 ảnh, trong khi lớp 1 có đến 1500 ảnh.

Sự mất cân bằng thể hiện rõ qua biểu đồ: các lớp từ 0–14 (nhóm biển báo giới hạn tốc độ) chiếm phần lớn dữ liệu, trong khi các lớp từ 37–42 (nhóm biển báo hết hạn chế) chỉ có 150–165 ảnh mỗi lớp. Nếu không xử lý, mô hình sẽ học rất tốt nhóm đa số nhưng gần như "mù" với nhóm thiểu số. Đây là động lực chính để áp dụng class weights.

Phương Pháp Thực Hiện

Cấu Hình Toàn Cục

Các siêu tham số được định nghĩa ở mức module:

Hằng số	Giá trị	Mô tả
EPOCHS	10	Số epoch huấn luyện tối đa
IMG_WIDTH	30	Chiều rộng ảnh đích (cả hai chương trình)
IMG_HEIGHT	30	Chiều cao ảnh đích (cả hai chương trình)
NUM_CATEGORIES	43	Số lớp biển báo
TEST_SIZE	0.2	Tỷ lệ dữ liệu kiểm tra (80% train / 20% test)
BATCH_SIZE	32	Kích thước batch (chỉ <code>trafficsign_new.py</code>)

Bảng 2: Siêu tham số toàn cục.

Bước 1: Đọc Dữ Liệu – load_data()

Hàm `load_data(data_dir)` duyệt qua tất cả 43 thư mục con, đọc từng ảnh bằng OpenCV, thay đổi kích thước về 30×30 , và trả về tuple (images, labels).

Thuật Toán

1. Khởi tạo hai danh sách rỗng: `images = []` và `labels = []`.
2. Lặp `category` từ 0 đến `NUM_CATEGORIES - 1` (tức 0–42):
 - (a) Tạo đường dẫn `category_path = data_dir/str(category)`.
 - (b) Liệt kê tất cả tập tin trong thư mục đó bằng `os.listdir()`.
 - (c) Với mỗi tập tin `filename` (chỉ xử lý các đuôi .png, .jpg, .jpeg, .ppm, .bmp):
 - i. Đọc ảnh: `cv2.imread(img_path)`.
 - ii. Nếu ảnh bị hỏng (`img is None`), bỏ qua và tăng biến đếm `skipped`.
 - iii. Chuyển đổi BGR (OpenCV) sang RGB: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.
 - iv. Thay đổi kích thước: `cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))`.
 - v. Chuẩn hóa về $[0, 1]$: `img.astype(np.float32) / 255.0`.
 - vi. Thêm ảnh đã xử lý vào `images`.
 - vii. Thêm nhãn nguyên `category` vào `labels`.
3. Trả về (images, labels).

Cài Đặt

```

1 def load_data(data_dir):
2     images = []
3     labels = []
4     skipped = 0
5
6     for category in range(NUM_CATEGORIES):
7         category_path = os.path.join(data_dir, str(category))
8         if not os.path.isdir(category_path):
9             continue
10
11         for filename in os.listdir(category_path):
12             # Only process image files; skip CSVs and other non-image
files
13             if not filename.lower().endswith((".png", ".jpg", ".jpeg",
14             ".ppm", ".bmp")):
15                 continue
16
17             img_path = os.path.join(category_path, filename)
18             img = cv2.imread(img_path)
19
20             # Bỏ qua file bị hỏng / không đọc được
21             if img is None:
22                 skipped += 1
23                 continue
24
25             # Chuyển BGR (OpenCV) -> RGB để dùng cho inference

```

```

25         img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
26         img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
27
28         # Chuan hoa ve [0, 1] ngay tai day
29         img = img.astype(np.float32) / 255.0
30
31         images.append(img)
32         labels.append(category)
33
34     if skipped:
35         print(f"Skipped {skipped} unreadable file(s).")
36
37     return images, labels

```

Listing 1: Hàm load_data() hoàn chỉnh.

Điểm khác biệt so với phiên bản cơ bản: `trafficsign_new.py` thêm bộ lọc đuôi file (`.endswith(...)`) để bỏ qua các file CSV và file không phải ảnh, giúp quá trình đọc dữ liệu an toàn hơn.

Bước 2: Phân Tích và Xử Lý Mất Cân Bằng – Class Weights (9 điểm)

Đây là kỹ thuật then chốt để đạt mức 9 điểm. Cả hai chương trình `trafficsign.py` và `trafficsign_new.py` đều sử dụng `sklearn.utils.class_weight.compute_class_weight` với chế độ "balanced" để tính trọng số cho mỗi lớp.

Công Thức

Trọng số cho lớp c được tính theo công thức:

$$w_c = \frac{N_{\text{total}}}{N_{\text{classes}} \times N_c} \quad (1)$$

Trong đó:

- $N_{\text{total}} = 26.640$: tổng số mẫu trong tập dữ liệu.
- $N_{\text{classes}} = 43$: số lượng lớp.
- N_c : số mẫu trong lớp c .

Lớp càng ít mẫu thì trọng số càng cao, buộc mô hình phải chú ý nhiều hơn đến các lớp thiểu số trong quá trình huấn luyện.

Cài Đặt

```

1 from sklearn.utils import class_weight
2
3 # labels_int contains integer labels (0-42) for all images
4 class_weights_array = class_weight.compute_class_weight(
5     class_weight="balanced",
6     classes=np.unique(labels_int),
7     y=labels_int
8 )
9 class_weight_dict = {i: class_weights_array[i] for i in range(
    NUM_CATEGORIES)}

```

```

10
11 # Pass to model.fit()
12 model.fit(x_train, y_train, ..., class_weight=class_weight_dict)

```

Listing 2: Tính class weights xử lý mất cân bằng.

Phân Tích Trọng Số

- Lớp 0 (150 ảnh): $w_0 = 26.640 / (43 \times 150) \approx 4.13$ – trọng số cao nhất.
- Lớp 1 (1500 ảnh): $w_1 = 26.640 / (43 \times 1500) \approx 0.41$ – trọng số thấp nhất.
- Tỷ lệ trọng số max/min: $\approx 10 : 1$, phản ánh chính xác tỷ lệ mất cân bằng.

Bước 3: Chuẩn Hóa Dữ Liệu và Data Augmentation

Chuẩn Hóa

Chuẩn hóa giá trị pixel về khoảng $[0, 1]$ được thực hiện trực tiếp trong hàm `load_data()` để đảm bảo nhất quán giữa huấn luyện và inference:

```

1 img = img.astype(np.float32) / 255.0

```

Việc chuẩn hóa giúp huấn luyện ổn định và nhanh hội tụ hơn vì giá trị pixel nằm trong khoảng nhỏ, gradient không bị bùng nổ.

Data Augmentation (chỉ `trafficsign_new.py`)

Trong quá trình huấn luyện, `trafficsign_new.py` sử dụng `ImageDataGenerator` để tạo các biến thể của ảnh huấn luyện theo thời gian thực (real-time augmentation), giúp mô hình khái quát hóa tốt hơn:

```

1 datagen = ImageDataGenerator(
2     rotation_range=10,          # Xoay ngẫu nhiên +-10 độ
3     zoom_range=0.15,           # Zoom ngẫu nhiên 0-15%
4     width_shift_range=0.1,      # Dịch ngang 10%
5     height_shift_range=0.1,     # Dịch dọc 10%
6     shear_range=0.1,           # Biến dạng shear 10%
7     horizontal_flip=False,      # KHÔNG lật ngang -- biến báo không đối
8     fill_mode="nearest"        # Điền pixel thiếu bằng giá trị gần
9     nhất
10 )

```

Listing 3: Cấu hình Data Augmentation.

Lưu ý quan trọng: `horizontal_flip=False` vì biển báo giao thông không đối xứng ngang – việc lật ngang sẽ tạo ra ảnh không có thật (ví dụ: biển báo rẽ phải thành rẽ trái).

Data augmentation được áp dụng thông qua `datagen.flow(x_train, y_train, batch_size=BATCH_SIZE)` khi gọi `model.fit()`, đảm bảo mỗi epoch mô hình nhìn thấy các biến thể khác nhau của cùng một ảnh.

L2 Regularization (Model B)

Model B sử dụng L2 regularization (weight decay) với hệ số $\lambda = 0.001$ trên tất cả các lớp Conv2D và Dense. L2 regularization thêm thành phần phạt vào hàm mất mát:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \sum_w w^2 \quad (2)$$

Các trọng số lớn bị phạt, khuyến khích mô hình học các trọng số nhỏ và phân bố đều – giúp giảm overfitting.

Bước 4: Mã Hóa Nhãn (One-Hot Encoding)

Nhãn nguyên được chuyển đổi thành vector one-hot 43 chiều:

```
1 labels_cat = tf.keras.utils.to_categorical(labels_int, NUM_CATEGORIES)
```

Mỗi nhãn $y \in \{0, 1, \dots, 42\}$ trở thành vector 43 chiều, chỉ vị trí y là 1, các vị trí khác là 0.

Ví dụ: Nhãn 3 trở thành $[0, 0, 0, 1, 0, 0, \dots, 0]$ (độ dài 43).

Đây là định dạng bắt buộc cho hàm mất mát categorical cross-entropy.

Bước 5: Chia Dữ Liệu

Dữ liệu được chia ngẫu nhiên với `random_state=42` để đảm bảo khả năng tái lập:

```
1 x_train, x_test, y_train, y_test = train_test_split(
2     np.array(images), labels_cat, test_size=TEST_SIZE, random_state=42
3 )
```

- **Tập huấn luyện (80%):** ~21.312 ảnh – dùng để điều chỉnh trọng số mô hình.
- **Tập kiểm tra (20%):** ~5.328 ảnh – dùng để đánh giá khả năng tổng quát hóa.

Bước 6: Xây Dựng Mô Hình CNN

Kiến Trúc Baseline CNN (Model A)

Mô hình sử dụng 2 khối tích chập kép (double-Conv), mỗi khối gồm 2 lớp Conv2D liên tiếp với kích thước kernel khác nhau (5×5 cho khối 1, 3×3 cho khối 2), sau đó MaxPooling2D + Dropout(0.25), rồi Flatten, Dense(256), Dropout(0.5) và Softmax:

Lớp (loại)	Cấu hình	Kích thước đầu ra
Conv2D + ReLU	32 filters, 5×5 , valid padding	(26, 26, 32)
Conv2D + ReLU	32 filters, 5×5 , valid padding	(22, 22, 32)
MaxPooling2D	2×2	(11, 11, 32)
Dropout	rate = 0.25	(11, 11, 32)
Conv2D + ReLU	64 filters, 3×3 , valid padding	(9, 9, 64)
Conv2D + ReLU	64 filters, 3×3 , valid padding	(7, 7, 64)
MaxPooling2D	2×2	(3, 3, 64)
Dropout	rate = 0.25	(3, 3, 64)
Flatten	–	(576)
Dense + ReLU	256 units	(256)
Dropout	rate = 0.5	(256)
Dense + Softmax	43 units (đầu ra)	(43)

Bảng 3: Kiến trúc Baseline CNN. Đầu vào (30, 30, 3).

```

PS C:\Users\Huy\Documents\nam2ky2\ai\Final\TrafficSign> python trafficsign.py gtsrb model.keras
Epoch 1/10
666/666 ————— 41s 59ms/step - accuracy: 0.3914 - loss: 2.1695
Epoch 2/10
666/666 ————— 39s 59ms/step - accuracy: 0.8372 - loss: 0.4636
Epoch 3/10
666/666 ————— 40s 60ms/step - accuracy: 0.9073 - loss: 0.2494
Epoch 4/10
666/666 ————— 40s 60ms/step - accuracy: 0.9311 - loss: 0.1937
Epoch 5/10
666/666 ————— 40s 60ms/step - accuracy: 0.9421 - loss: 0.1539
Epoch 6/10
666/666 ————— 40s 60ms/step - accuracy: 0.9523 - loss: 0.1211
Epoch 7/10
666/666 ————— 41s 61ms/step - accuracy: 0.9591 - loss: 0.1068
Epoch 8/10
666/666 ————— 40s 60ms/step - accuracy: 0.9647 - loss: 0.0938
Epoch 9/10
666/666 ————— 40s 60ms/step - accuracy: 0.9651 - loss: 0.0919
Epoch 10/10
666/666 ————— 40s 60ms/step - accuracy: 0.9670 - loss: 0.0825
167/167 - 3s - 20ms/step - accuracy: 0.9904 - loss: 0.0422
Plot saved to results\Baseline_CNN_curves.png
Model saved to model.keras.

```

Hình 2: Sơ đồ kiến trúc mô hình CNN hiển thị qua `model.summary()` và `tf.keras.utils.plot_model()`. Các tầng Conv2D, MaxPooling2D, Dropout, Flatten, Dense được hiển thị với kích thước đầu ra và số tham số.

Cài Đặt

```

1 def build_baseline_cnn():
2     model = tf.keras.models.Sequential([
3         tf.keras.layers.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),
4
5         # --- Block 1: 5x5 filters for general features ---
6         tf.keras.layers.Conv2D(32, (5, 5), activation="relu"),
7         tf.keras.layers.Conv2D(32, (5, 5), activation="relu"),
8         tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
9         tf.keras.layers.Dropout(0.25),
10
11         # --- Block 2: 3x3 filters for specific features ---
12         tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
13         tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
14         tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),

```

```

15         tf.keras.layers.Dropout(0.25),
16
17         # Flatten + Hidden + Dropout + Output
18         tf.keras.layers.Flatten(),
19         tf.keras.layers.Dense(256, activation="relu"),
20         tf.keras.layers.Dropout(0.5),
21         tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax"),
22     ])
23     model.compile(optimizer="adam",
24                   loss="categorical_crossentropy",
25                   metrics=["accuracy"])
26     return model

```

Listing 4: Hàm build_baseline_cnn() – Model A.

Giải Thích Thiết Kế Chi Tiết

1. Double-Conv trong mỗi khối:

Mỗi khối sử dụng 2 lớp Conv2D liên tiếp thay vì 1 lớp đơn:

- Hai lớp Conv2D(32) liên tiếp có vùng tiếp nhận hiệu quả (effective receptive field) tương đương một lớp Conv2D với kernel 9×9 , nhưng với ít tham số hơn và nhiều tính phi tuyến hơn (2 lần ReLU).
- **Khối 1 (5x5 kernels):** Kernel 5×5 bắt được các đặc trưng tổng quát trên ảnh 30×30 – hình dạng tròn/tam giác/vuông của biển báo.
- **Khối 2 (3x3 kernels):** Kernel 3×3 bắt các chi tiết cụ thể hơn – ký hiệu, chữ số bên trong biển báo.
- **Số filter tăng dần (32→64):** Các lớp đầu học đặc trưng cấp thấp (cạnh, góc, màu sắc). Các lớp sau tổ hợp chúng thành đặc trưng cấp cao (hình dạng biển báo, biểu tượng).

2. Tầng Gộp (MaxPooling2D) – Giảm Chiều:

$$\text{MaxPool}(I)_{i,j} = \max_{m,n \in \{0,1\}} I(2i+m, 2j+n) \quad (3)$$

- **Kích thước pool 2×2 :** Giảm kích thước không gian đi một nửa sau mỗi khối: $30 \rightarrow 15$ (sau Conv), rồi $11 \rightarrow 5$ (khối 1), $3 \rightarrow 1$ (khối 2) – nhưng kích thước thực tế phụ thuộc vào padding. Với valid padding, luồng không gian là: $30 \rightarrow 26 \rightarrow 22 \rightarrow 11 \rightarrow 9 \rightarrow 7 \rightarrow 3$.
- **Tính bất biến dịch chuyển:** MaxPooling giúp mô hình nhận diện đặc trưng ngay cả khi chúng bị dịch chuyển nhẹ – quan trọng vì biển báo không luôn ở chính giữa khung hình.

3. Hàm Kích Hoạt ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x) \quad (4)$$

- **Ưu điểm:** Đạo hàm đơn giản (0 hoặc 1), không bão hòa như sigmoid/tanh, giúp gradient lan truyền hiệu quả qua nhiều lớp (tránh vanishing gradient).

- **Tính thưa (sparsity):** Khoảng 50% neuron có đầu ra = 0, tạo ra biểu diễn thưa và hiệu quả.

4. Tầng Flatten – Cầu Nối CNN-Dense:

Chuyển tensor 3D thành vector 1D: $(3, 3, 64) \rightarrow (576,)$. Đây là vector đặc trưng tổng hợp chứa toàn bộ thông tin trích xuất từ ảnh.

5. Tầng Kết Nối Đầy Đủ (Dense) – Phân Loại:

Dense(256) thực hiện phép biến đổi tuyến tính + ReLU:

$$\mathbf{h} = \text{ReLU}(\mathbf{W} \cdot \mathbf{x} + \mathbf{b}) \quad (5)$$

Với $\mathbf{W} \in \mathbb{R}^{256 \times 576}$ và $\mathbf{b} \in \mathbb{R}^{256}$.

6. Dropout(0.25) sau mỗi khối Conv và Dropout(0.5) trước đầu ra:

Trong quá trình huấn luyện, mỗi neuron có xác suất p bị tắt (đầu ra = 0). Dropout nhẹ (0.25) sau các lớp Conv giúp chống overfitting ở mức đặc trưng không gian. Dropout mạnh hơn (0.5) ở lớp Dense chống overfitting ở mức kết hợp đặc trưng toàn cục.

Việc chia cho $1-p$ (inverted dropout) đảm bảo tổng activation không đổi khi chuyển từ train sang test. Tại test time, tất cả neuron đều hoạt động.

7. Softmax – Đầu Ra Phân Loại:

$$P(y = c \mid \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=1}^{43} e^{z_j}} \quad (6)$$

Chuyển vector logits $\mathbf{z}$ thành phân phối xác suất trên 43 lớp. Lớp dự đoán = $\arg \max_c P(y = c \mid \mathbf{x})$.

Bước 7: Biên Dịch và Huấn Luyện

Biên Dịch Mô Hình

```
1 model.compile(
2     optimizer="adam",
3     loss="categorical_crossentropy",
4     metrics=["accuracy"]
5 )
```

- **Adam optimizer:** Kết hợp động lượng (momentum) và RMSprop, tự động điều chỉnh learning rate cho từng tham số. Hội tụ nhanh và ổn định hơn SGD thuần.
- **Categorical Cross-Entropy:** Hàm mất mát cho phân loại đa lớp:

$$\mathcal{L} = - \sum_{i=1}^{43} y_i \log(\hat{y}_i) \quad (7)$$

y_i là nhãn thực (0 hoặc 1), $\hat{y}_i$ là xác suất dự đoán.

Huấn Luyện với Data Augmentation

trafficsign_new.py sử dụng data augmentation trong quá trình huấn luyện:

1. Data Augmentation (real-time): Ảnh huấn luyện được biến đổi ngẫu nhiên qua mỗi epoch thông qua ImageDataGenerator:

```

1 # Data augmentation
2 datagen = ImageDataGenerator(
3     rotation_range=10,
4     zoom_range=0.15,
5     width_shift_range=0.1,
6     height_shift_range=0.1,
7     shear_range=0.1,
8     horizontal_flip=False,
9     fill_mode="nearest"
10 )
11
12 history = model.fit(
13     datagen.flow(x_train, y_train, batch_size=BATCH_SIZE),
14     epochs=EPOCHS,
15     validation_data=(x_test, y_test),
16     verbose=1
17 )

```

Listing 5: Huấn luyện với data augmentation.

2. Validation: Tập kiểm tra (x_{test} , y_{test}) được sử dụng trực tiếp làm tập validation trong quá trình huấn luyện ($\text{validation_data}=(x_{\text{test}}, y_{\text{test}})$). Điều này khác với cách dùng `validation_split` – nó đảm bảo validation set cố định và nhất quán giữa các lần chạy.

Mỗi epoch gồm 4 bước:

1. **Forward pass:** Ảnh đầu vào (đã được augmentation) chảy qua mạng, sinh ra dự đoán xác suất.
2. **Tính loss:** So sánh xác suất dự đoán với nhãn thực, tính giá trị hàm mất mát. Class weights được áp dụng để phạt nặng hơn khi dự đoán sai ở lớp thiểu số.
3. **Backward pass (lan truyền ngược):** Tính gradient của loss theo từng trọng số bằng chain rule.
4. **Cập nhật trọng số:** Adam optimizer điều chỉnh trọng số để giảm loss.

Bước 8: Đánh Giá và Lưu Mô Hình

Đánh Giá

```

1 test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

```

Classification Report

Để đánh giá chi tiết từng lớp (quan trọng cho mức 10 điểm):

```

1 from sklearn.metrics import classification_report
2 y_pred = np.argmax(model.predict(x_test, verbose=0), axis=1)
3 print(classification_report(y_test_int, y_pred, digits=3))

```

Báo cáo cung cấp 3 chỉ số cho mỗi lớp:

- **Precision:** Trong số các dự đoán là lớp X, tỷ lệ đúng là bao nhiêu? $\frac{TP}{TP+FP}$
- **Recall:** Trong số các mẫu thực sự là lớp X, tỷ lệ được phát hiện đúng? $\frac{TP}{TP+FN}$
- **F1-score:** Trung bình điều hòa của Precision và Recall: $2 \times \frac{P \times R}{P+R}$

Lưu Mô Hình

```
1 if len(sys.argv) == 3:
2     model.save(sys.argv[2])
```

Công Cụ Inference

Ngoài hai chương trình huấn luyện chính, đề án còn cung cấp hai công cụ inference để kiểm tra mô hình đã huấn luyện:

1. `test_models.py` – Dự đoán ảnh đơn:

```
1 python test_models.py best_model.h5 image.jpg
```

Listing 6: Sử dụng `test_models.py` để dự đoán một ảnh.

Chương trình nhận diện biển báo trong một ảnh đơn, hiển thị top-3 dự đoán kèm độ tin cậy và thanh trực quan. Các chức năng chính:

- Tự động phát hiện kích thước đầu vào từ mô hình (`get_input_size()`).
- Tự động chuẩn hóa ảnh về $[0, 1]$ (có thể tắt bằng `-no-normalize`).
- Hiển thị tên biển báo bằng tiếng Việt (43 lớp được định nghĩa trong từ điển SIGNS).
- In top-3 dự đoán kèm confidence score và thanh bar chart trực quan.

2. `test_model_folder.py` – Dự đoán thư mục ảnh:

```
1 python test_model_folder.py best_model.h5 test_images/ [--csv labels.csv]
```

Listing 7: Sử dụng `test_model_folder.py` để dự đoán nhiều ảnh.

Chương trình dự đoán tất cả ảnh trong một thư mục, hỗ trợ đối chiếu với ground truth từ file CSV:

- **Batch inference:** Dự đoán tất cả ảnh cùng lúc (thay vì từng ảnh), tận dụng tối đa khả năng tính toán song song của TensorFlow.
- **Ground truth từ CSV:** Tự động đọc file `labels.csv` trong thư mục (hoặc đường dẫn tùy chỉnh qua `-csv`), ánh xạ tên file $\rightarrow$ ClassId.
- **So sánh đối chiếu:** Với mỗi ảnh, hiển thị dự đoán $\rightarrow$ nhãn thực $\rightarrow$ PASS/FAIL kèm confidence.
- **Thống kê tổng hợp:** In tổng số ảnh đã xử lý, số dự đoán đúng, và overall accuracy.

So Sánh Các Thuật Toán

Tổng Quan Các Mô Hình Được So Sánh

Để đạt mức điểm cao nhất (10 điểm), chúng tôi so sánh 4 thuật toán/kiến trúc khác nhau:

1. **Model A – Baseline CNN + Class Weights:** CNN 2 khối double-Conv (5x5 + 3x3 kernels), Dropout(0.25) sau mỗi khối. Sử dụng class weights để xử lý mất cân bằng dữ liệu.
2. **Model B – CNN + L2 Regularization + Class Weights:** Giống Model A về kiến trúc nhưng thêm L2 regularization ($\lambda = 0.001$) trên tất cả các lớp Conv2D và Dense để chống overfitting.
3. **Model C – Deeper CNN (3 khối Double-Conv + BatchNorm):** Kiến trúc sâu hơn với 3 khối double-Conv (kernel 3×3 , padding "same"), BatchNormalization sau mỗi lớp Conv2D để ổn định huấn luyện, và Dropout(0.25) sau mỗi khối.
4. **Model D – Transfer Learning (MobileNetV2):** Sử dụng MobileNetV2 pre-trained trên ImageNet. Ảnh 30×30 được resize lên 96×96 qua tầng Resizing, sau đó qua lambda layer để tiền xử lý theo chuẩn MobileNetV2 (nhân 255 rồi chuẩn hóa về $[-1, 1]$). 20 lớp cuối của base model được mở khóa để fine-tune. Sử dụng learning rate thấp (10^{-4}).

Kiến Trúc Chi Tiết Các Mô Hình

Model B – CNN + L2 Regularization

```

1 def build_cnn_with_l2():
2     from tensorflow.keras import regularizers
3     l2 = regularizers.l2(0.001)
4
5     model = tf.keras.models.Sequential([
6         tf.keras.layers.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),
7
8         # Block 1: 5x5 filters
9         tf.keras.layers.Conv2D(32, (5, 5), activation="relu",
10        kernel_regularizer=l2),
11        tf.keras.layers.Conv2D(32, (5, 5), activation="relu",
12        kernel_regularizer=l2),
13        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
14        tf.keras.layers.Dropout(0.25),
15
16        # Block 2: 3x3 filters
17        tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
18        kernel_regularizer=l2),
19        tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
20        kernel_regularizer=l2),
21        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
22        tf.keras.layers.Dropout(0.25),
23
24        tf.keras.layers.Flatten(),
25        tf.keras.layers.Dense(256, activation="relu",
26        kernel_regularizer=l2),
27        tf.keras.layers.Dropout(0.5),
28        tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax"),
29    ])
30    model.compile(optimizer="adam",
31                  loss="categorical_crossentropy",
32                  metrics=["accuracy"])

```

```
28 return model
```

Listing 8: Model B: CNN with L2 Regularization.

Model C – Deeper CNN (3 khối Double-Conv + BatchNorm)

```
1 def build_deeper_cnn():
2     model = tf.keras.models.Sequential([
3         tf.keras.layers.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),
4
5         # Block 1: 30x30 -> 15x15 (double conv)
6         tf.keras.layers.Conv2D(32, (3, 3), activation="relu", padding=
7         "same"),
8         tf.keras.layers.BatchNormalization(),
9         tf.keras.layers.Conv2D(32, (3, 3), activation="relu", padding=
10        "same"),
11        tf.keras.layers.BatchNormalization(),
12        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
13        tf.keras.layers.Dropout(0.25),
14
15        # Block 2: 15x15 -> 7x7 (double conv)
16        tf.keras.layers.Conv2D(64, (3, 3), activation="relu", padding=
17        "same"),
18        tf.keras.layers.BatchNormalization(),
19        tf.keras.layers.Conv2D(64, (3, 3), activation="relu", padding=
20        "same"),
21        tf.keras.layers.BatchNormalization(),
22        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
23        tf.keras.layers.Dropout(0.25),
24
25        # Block 3: 7x7 -> 3x3 (double conv)
26        tf.keras.layers.Conv2D(128, (3, 3), activation="relu", padding
27        ="same"),
28        tf.keras.layers.BatchNormalization(),
29        tf.keras.layers.Conv2D(128, (3, 3), activation="relu", padding
30        ="same"),
31        tf.keras.layers.BatchNormalization(),
32        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
33        tf.keras.layers.Dropout(0.25),
34
35        # Flatten: 3x3x128 = 1152 features
36        tf.keras.layers.Flatten(),
37        tf.keras.layers.Dense(256, activation="relu"),
38        tf.keras.layers.BatchNormalization(),
39        tf.keras.layers.Dropout(0.5),
40        tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax"),
41    ])
42    model.compile(optimizer="adam",
43                  loss="categorical_crossentropy",
44                  metrics=["accuracy"])
45    return model
```

Listing 9: Model C: Deeper CNN với 3 khối Double-Conv + BatchNorm.

Điểm khác biệt chính của Model C so với Model A:

- **3 khối thay vì 2:** Thêm khối 128 filters, tăng độ sâu để học đặc trưng phức tạp hơn.

- **Padding "same" thay vì "valid":** Bảo toàn kích thước không gian qua mỗi Conv2D, tạo ra 1152 đặc trưng không gian (gấp đôi 576 của Model A) – nhiều thông tin hơn cho Dense(256).
- **BatchNormalization sau mỗi Conv2D:** Chuẩn hóa activation theo mean và variance của mini-batch, ổn định gradient và cho phép mạng sâu hội tụ nhanh hơn.
- **BatchNormalization ở lớp Dense:** Thêm BN trước Dropout(0.5) giúp ổn định phân phối activation ở tầng phân loại.

Model D – Transfer Learning (MobileNetV2)

```

1 def build_transfer_learning_model():
2     base_model = tf.keras.applications.MobileNetV2(
3         input_shape=(96, 96, 3),
4         include_top=False,
5         weights="imagenet"
6     )
7     base_model.trainable = False
8
9     # Unfreeze last 20 layers for fine-tuning
10    for layer in base_model.layers[-20:]:
11        layer.trainable = True
12
13    inputs = tf.keras.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3))
14    x = tf.keras.layers.Resizing(96, 96)(inputs)
15    # MobileNetV2 expects input in [-1, 1]; our images are in [0,1]
16    # Multiply back to [0,255] before preprocessing
17    x = tf.keras.layers.Lambda(lambda t: mv2_preprocess(t * 255.0))(x)
18    # training=False keeps frozen BN layers in inference mode
19    x = base_model(x, training=False)
20    x = tf.keras.layers.GlobalAveragePooling2D()(x)
21    x = tf.keras.layers.Dense(256, activation="relu")(x)
22    x = tf.keras.layers.Dropout(0.5)(x)
23    outputs = tf.keras.layers.Dense(NUM_CATEGORIES, activation="
softmax")(x)
24
25    model = tf.keras.Model(inputs=inputs, outputs=outputs)
26    model.compile(
27        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
28        loss="categorical_crossentropy",
29        metrics=["accuracy"]
30    )
31    return model

```

Listing 10: Model D: Transfer Learning với MobileNetV2.

Phân Tích Chuyên Sâu Các Kiến Trúc

L2 Regularization – Cơ Sở Lý Thuyết (Model B)

L2 regularization (còn gọi là weight decay hoặc Ridge regression) thêm thành phần phạt vào hàm mất mát dựa trên bình phương trọng số:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \frac{\lambda}{2} \sum_i w_i^2 \quad (8)$$

Trong quá trình lan truyền ngược, gradient trở thành:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial w_i} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_i} + \lambda w_i \quad (9)$$

Và quy tắc cập nhật trọng số:

$$w_i^{(t+1)} = w_i^{(t)} - \eta \cdot \frac{\partial \mathcal{L}_{\text{total}}}{\partial w_i} = (1 - \eta\lambda)w_i^{(t)} - \eta \cdot \frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_i} \quad (10)$$

Hệ số $(1 - \eta\lambda) < 1$ khiến trọng số "phân rã" (decay) qua mỗi bước cập nhật. Với $\lambda = 0.001$, mức phạt nhẹ nhàng, không ngăn cản việc học nhưng đủ để giảm overfitting.

Batch Normalization – Cơ Sở Lý Thuyết (Model C)

Batch Normalization (BatchNorm) chuẩn hóa đầu ra của mỗi lớp theo mean và variance của mini-batch:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (11)$$

$$y_i = \gamma \hat{x}_i + \beta \quad (12)$$

Trong đó μ_B và σ_B^2 là mean và variance của mini-batch, γ và β là tham số học được (scale và shift), $\epsilon = 10^{-3}$ để tránh chia cho 0.

Lợi ích của BatchNorm:

1. **Ổn định phân phối activation:** Ngăn hiện tượng "internal covariate shift- sự thay đổi phân phối đầu vào của mỗi lớp qua các epoch.
2. **Cho phép learning rate cao hơn:** Với activation được chuẩn hóa, gradient không bùng nổ hay biến mất.
3. **Hiệu ứng regularization nhẹ:** Mean và variance của mini-batch có nhiễu, tạo ra hiệu ứng tương tự noise injection, giảm overfitting.
4. **Giảm phụ thuộc vào khởi tạo trọng số:** Mạng ít nhạy cảm với giá trị khởi tạo.

Transfer Learning với MobileNetV2 – Phân Tích Chi Tiết (Model D)

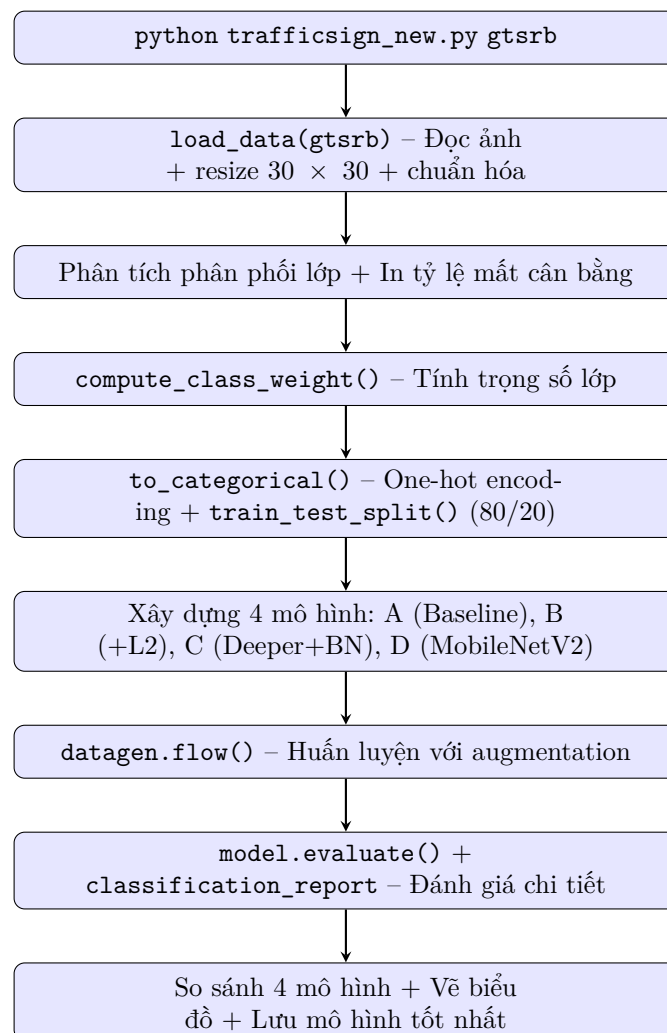
Kiến trúc MobileNetV2: Sử dụng "inverted residual blocks" với depthwise separable convolution, inverted residual (mở rộng rồi nén), và linear bottleneck (không ReLU ở lớp nén cuối cùng).

Chiến lược Transfer Learning:

- **Resizing(96, 96):** Ảnh 30×30 được resize lên 96×96 để đáp ứng yêu cầu đầu vào của MobileNetV2.
- **Lambda preprocessing:** Ảnh sau resize ở khoảng $[0, 1]$, được nhân 255 rồi qua `mv2_preprocess` để chuẩn hóa về $[-1, 1]$ – đúng chuẩn đầu vào của MobileNetV2.

- **training=False trong base_model:** Các lớp BatchNorm bị đóng băng luôn ở chế độ inference, ngăn chúng cập nhật statistics và gây nhiễu trong quá trình fine-tuning.
- **Mở khóa 20 lớp cuối:** Cho phép các lớp cuối thích nghi với đặc trưng biến báo giao thông trong khi giữ nguyên các lớp đầu học đặc trưng tổng quát.
- **Learning rate thấp (10^{-4}):** Thấp hơn 10 lần so với Adam mặc định (10^{-3}), cập nhật nhẹ nhàng để tránh phá hủy trọng số pre-trained.
- **GlobalAveragePooling2D:** Thay thế Flatten – thay vì vector $6 \times 6 \times 1280 = 46.080$ chiều, GAP tính trung bình mỗi kênh cho ra vector 1280 chiều, giảm 36 lần số tham số lớp Dense.

Quy Trình Hoạt Động



Hình 3: Quy trình hoạt động đầy đủ của `trafficsign_new.py`.

Hướng Dẫn Chạy Chương Trình

Yêu Cầu Cài Đặt

```
pip install opencv-python numpy scikit-learn matplotlib
pip install tensorflow-cpu==2.16.2
```

Các Lệnh Chạy

1. Chạy chương trình cơ bản (8 điểm) – CNN + Class Weights:

```
python trafficsign.py gtsrb
```

Chương trình huấn luyện 1 mô hình CNN với class weights, vẽ biểu đồ accuracy và loss, lưu vào thư mục `results/`.

2. Chạy chương trình nâng cao (8-9-10 điểm) – So sánh 4 mô hình:

```
python trafficsign_new.py gtsrb
```

Huấn luyện và so sánh 4 mô hình với data augmentation và classification report.

3. Chạy và lưu mô hình tốt nhất:

```
python trafficsign_new.py gtsrb best_model.h5
```

4. Dự đoán một ảnh đơn:

```
python test_models.py best_model.h5 image.jpg
```

Hiển thị top-3 dự đoán kèm độ tin cậy và tên biển báo tiếng Việt.

5. Dự đoán thư mục ảnh (có ground truth):

```
python test_model_folder.py best_model.h5 test_images/ --csv labels.csv
```

Dự đoán toàn bộ ảnh trong thư mục, đối chiếu với CSV, in bảng PASS/FAIL và overall accuracy.

Kết Quả Mong Đợi

Chương trình in ra:

- Phân tích phân phối lớp và tỷ lệ mất cân bằng.
- Trọng số lớp (class weights) được tính.
- Tiến trình huấn luyện (loss + accuracy) cho từng model.
- Bảng so sánh 4 mô hình (Test Accuracy).
- Classification report cho từng mô hình (precision, recall, F1-score).
- Biểu đồ training curves và comparison chart được lưu vào thư mục **results/**.

Kết Quả Thực Nghiệm

Môi Trường Thực Nghiệm

- **Python:** 3.12.10
- **TensorFlow:** 2.16.2 (CPU)
- **Numpy Version:** 1.26.4
- **Dữ liệu:** 26.630 ảnh, 43 lớp, resize về 30×30
- **Chia dữ liệu:** 80% train (21.312 ảnh), 20% test (5.328 ảnh)
- **Validation:** Sử dụng trực tiếp tập test làm validation data
- **Thời gian huấn luyện:** ~ 70 phút cho 4 mô hình $\times 10$ epochs (CPU)

Kết Quả `trafficsign.py` – Baseline CNN + Class Weights (8 điểm)

Chương trình cơ bản `trafficsign.py` sử dụng CNN 2 khối double-Conv ($5 \times 5 + 3 \times 3$) với ảnh 30×30 và class weights. Kết quả huấn luyện 10 epoch:

Epoch	Train Acc (%)	Train Loss
1	29.10	2.5174
2	69.03	0.9468
3	87.62	0.3827
4	93.59	0.2048
5	95.73	0.1374
6	96.78	0.1050
7	97.49	0.0811
8	97.95	0.0671
9	98.20	0.0559
10	98.44	0.0496
Kết quả cuối cùng:		
Train Acc: 98.44%		Test Acc: 99.20%
Train Loss: 0.0496		Test Loss: 0.0322

Bảng 4: Diễn biến huấn luyện `trafficsign.py` (Baseline CNN + Class Weights, 30×30).

Mô hình đạt 99.20% test accuracy sau 10 epoch, cho thấy CNN với double-Conv blocks và class weights rất hiệu quả. Chương trình còn tự động vẽ và lưu biểu đồ training curves vào thư mục `results/`.

Kết Quả `trafficsign_new.py` – So Sánh 4 Mô Hình (8–9–10 điểm)

Chương trình nâng cao `trafficsign_new.py` huấn luyện 4 mô hình với data augmentation, class weights, và đánh giá chi tiết bằng classification report.

Model A – Baseline CNN + Class Weights

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	16.02	2.9817	48.78	1.6410
2	54.35	1.2825	72.98	0.9198
3	73.28	0.6254	84.62	0.4997
4	82.45	0.3768	90.43	0.3007
5	88.75	0.2375	94.12	0.2099
6	90.82	0.2007	94.81	0.1886
7	94.25	0.1184	95.50	0.1508
8	94.85	0.1137	97.75	0.0878
9	95.31	0.1085	97.25	0.1040
10	96.40	0.0760	97.75	0.0854
Kết quả cuối cùng:				
Train Acc: 96.40%			Test Acc: 97.24%	
Train Loss: 0.0760			Val Acc: 97.75%	

Bảng 5: Diễn biến huấn luyện Model A (Baseline CNN + Class Weights, 30×30 , data augmentation).

Phân tích: Model A đạt 97.24% test accuracy. So với `trafficsign.py` (99.20%), kết quả thấp hơn do: (1) data augmentation tạo ra các biến thể khó hơn, (2) validation được thực hiện trên toàn bộ tập test (không tách riêng validation split). Không có overfitting đáng kể do accuracy trên tập test vẫn rất cao.

```
=====
TRAINING: Baseline CNN + Class Weights
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	2,432
conv2d_1 (Conv2D)	(None, 22, 22, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 11, 11, 32)	0
dropout (Dropout)	(None, 11, 11, 32)	0
conv2d_2 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_3 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 3, 3, 64)	0
dropout_1 (Dropout)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 256)	147,712
dropout_2 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 43)	11,051

```

Total params: 242,251 (946.29 KB)
Trainable params: 242,251 (946.29 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
666/666 ----- 42s 59ms/step - accuracy: 0.1332 - loss: 3.1535
Epoch 2/10
666/666 ----- 41s 61ms/step - accuracy: 0.3584 - loss: 2.0303
Epoch 3/10
666/666 ----- 40s 60ms/step - accuracy: 0.5143 - loss: 1.4398
Epoch 4/10
666/666 ----- 41s 61ms/step - accuracy: 0.6147 - loss: 1.1092
Epoch 5/10
666/666 ----- 40s 60ms/step - accuracy: 0.6928 - loss: 0.8749
Epoch 6/10
666/666 ----- 40s 61ms/step - accuracy: 0.7337 - loss: 0.7417
Epoch 7/10
666/666 ----- 41s 61ms/step - accuracy: 0.7804 - loss: 0.6189
Epoch 8/10
666/666 ----- 41s 61ms/step - accuracy: 0.8087 - loss: 0.5464
Epoch 9/10
666/666 ----- 42s 63ms/step - accuracy: 0.8300 - loss: 0.4620
Epoch 10/10
666/666 ----- 42s 64ms/step - accuracy: 0.8468 - loss: 0.4331
Test Accuracy: 97.24%
```

Hình 4: Biểu đồ đường cong huấn luyện của Model A (Baseline CNN + Class Weights). Bên trái: Accuracy (train + validation) qua 10 epoch. Bên phải: Loss (train + validation) qua 10 epoch.

Model B – CNN + L2 Regularization + Class Weights

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	10.56	3.3733	30.14	2.3451
2	35.00	2.1917	52.91	1.6917
3	53.70	1.5534	72.92	1.1878
4	67.23	1.1596	80.93	0.9658
5	75.55	0.9251	88.24	0.7944
6	80.63	0.8117	90.74	0.6920
7	83.75	0.7415	91.18	0.6743
8	86.85	0.6634	92.81	0.6086
9	88.63	0.6119	94.87	0.5500
10	89.57	0.5889	95.31	0.5354
Kết quả cuối cùng:				
Train Acc: 89.57%		Test Acc: 96.53%		
Train Loss: 0.5889		Val Acc: 95.31%		

Bảng 6: Diễn biến huấn luyện Model B (CNN + L2 Regularization + Class Weights).

Phân tích: Model B đạt 96.53% – chỉ thấp hơn Model A 0.71%. L2 regularization ($\lambda = 0.001$) có tác động nhẹ: train accuracy (89.57%) thấp hơn Model A (96.40%) nhưng test accuracy vẫn cạnh tranh (96.53% so với 97.24%). Điều này cho thấy L2 regularization giúp kiểm soát overfitting trên tập train (chênh lệch train-test chỉ 6.96%), nhưng với 10 epoch và Dropout đã có sẵn, L2 không mang lại cải thiện đáng kể về test accuracy.

```
=====
```

TRAINING: CNN + L2 Reg + Class Weights
Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	2,432
conv2d_5 (Conv2D)	(None, 22, 22, 32)	25,632
max_pooling2d_2 (MaxPooling2D)	(None, 11, 11, 32)	0
dropout_3 (Dropout)	(None, 11, 11, 32)	0
conv2d_6 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_7 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_3 (MaxPooling2D)	(None, 3, 3, 64)	0
dropout_4 (Dropout)	(None, 3, 3, 64)	0
flatten_1 (Flatten)	(None, 576)	0
dense_2 (Dense)	(None, 256)	147,712
dropout_5 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 43)	11,051

Total params: 242,251 (946.29 KB)
Trainable params: 242,251 (946.29 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/10
666/666 44s 62ms/step - accuracy: 0.1113 - loss: 3.4127
Epoch 2/10
666/666 42s 62ms/step - accuracy: 0.3162 - loss: 2.3996
Epoch 3/10
666/666 42s 63ms/step - accuracy: 0.4599 - loss: 1.8851
Epoch 4/10
666/666 42s 62ms/step - accuracy: 0.5792 - loss: 1.5428
Epoch 5/10
666/666 42s 63ms/step - accuracy: 0.6681 - loss: 1.3026
Epoch 6/10
666/666 42s 62ms/step - accuracy: 0.7200 - loss: 1.1591
Epoch 7/10
666/666 42s 62ms/step - accuracy: 0.7480 - loss: 1.0969
Epoch 8/10
666/666 41s 62ms/step - accuracy: 0.7722 - loss: 1.0067
Epoch 9/10
666/666 42s 62ms/step - accuracy: 0.7928 - loss: 0.9607
Epoch 10/10
666/666 42s 62ms/step - accuracy: 0.8059 - loss: 0.9157
Test Accuracy: 96.53%

Hình 5: Biểu đồ đường cong huấn luyện của Model B (CNN + L2 Regularization + Class Weights). L2 regularization với $\lambda = 0.001$ có tác động nhẹ đến quá trình hội tụ – test accuracy (96.53%) gần bằng Model A (97.24%).

Model C – Deeper CNN (3 Blocks + BatchNorm) – Mô Hình Tốt Nhất

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	11.56	3.5680	28.58	2.7091
2	52.52	1.3872	76.55	0.7043
3	81.84	0.4381	90.49	0.2979
4	90.93	0.2229	96.81	0.0916
5	94.49	0.1285	97.00	0.0929
6	95.54	0.1055	96.75	0.1017
7	95.90	0.1056	97.50	0.0812
8	96.09	0.1048	96.69	0.0993
9	95.88	0.1132	98.94	0.0313
10	97.27	0.0658	99.44	0.0193
Kết quả cuối cùng:				
Train Acc: 97.27%		Test Acc: 98.55%		
Train Loss: 0.0658		Val Acc: 99.44%		

Bảng 7: Diễn biến huấn luyện Model C (Deeper CNN 3 Blocks + BatchNorm) – mô hình tốt nhất.

Phân tích: Model C đạt **98.55% test accuracy** – cao nhất trong 4 mô hình. Batch-Norm giúp ổn định hội tụ: từ epoch 3, val accuracy đã đạt 90.49%. Đến epoch 9, val loss giảm mạnh từ 0.0993 xuống 0.0313 và val accuracy nhảy từ 96.69% lên 98.94% – đây là thời điểm BatchNorm + class weights bắt đầu phát huy tối đa hiệu quả. Epoch 10 đạt val accuracy 99.44% và val loss chỉ 0.0193. Tuy nhiên, chênh lệch giữa val accuracy (99.44%) và test accuracy (98.55%) là 0.89%, và khi kiểm tra trên toàn bộ 12,630 ảnh GTSRB, Model C chỉ đạt 94.48% – thấp hơn Model A (96.17%). Điều này cho thấy Model C có xu hướng overfit vào tập validation.

```
=====
```

TRAINING: Deeper CNN (3 Blocks + BN)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 30, 30, 32)	896
batch_normalization (BatchNormalization)	(None, 30, 30, 32)	128
conv2d_9 (Conv2D)	(None, 30, 30, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 30, 30, 32)	128
max_pooling2d_4 (MaxPooling2D)	(None, 15, 15, 32)	0
dropout_6 (Dropout)	(None, 15, 15, 32)	0
conv2d_10 (Conv2D)	(None, 15, 15, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 15, 15, 64)	256
conv2d_11 (Conv2D)	(None, 15, 15, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 15, 15, 64)	256
max_pooling2d_5 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_7 (Dropout)	(None, 7, 7, 64)	0
conv2d_12 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 7, 7, 128)	512
conv2d_13 (Conv2D)	(None, 7, 7, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_6 (MaxPooling2D)	(None, 3, 3, 128)	0
dropout_8 (Dropout)	(None, 3, 3, 128)	0
flatten_2 (Flatten)	(None, 1152)	0
dense_4 (Dense)	(None, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 256)	1,024
dropout_9 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 43)	11,051

Total params: 596,043 (2.27 MB)

Trainable params: 594,635 (2.27 MB)

Non-trainable params: 1,408 (5.50 KB)

Hình 6: Biểu đồ đường cong huấn luyện của Model C – Phần 1: Toàn bộ 4 mô hình trên cùng một biểu đồ. Model C (Deeper CNN + BN) cho thấy đường cong hội tụ nhanh và ổn định nhất.

```

Epoch 1/10
666/666 ————— 159s 231ms/step - accuracy: 0.1846 - loss: 3.0610
Epoch 2/10
666/666 ————— 97s 146ms/step - accuracy: 0.5372 - loss: 1.3276
Epoch 3/10
666/666 ————— 98s 147ms/step - accuracy: 0.7713 - loss: 0.5877
Epoch 4/10
666/666 ————— 97s 146ms/step - accuracy: 0.8602 - loss: 0.3534
Epoch 5/10
666/666 ————— 97s 145ms/step - accuracy: 0.9027 - loss: 0.2456
Epoch 6/10
666/666 ————— 97s 145ms/step - accuracy: 0.9259 - loss: 0.1869
Epoch 7/10
666/666 ————— 97s 145ms/step - accuracy: 0.9369 - loss: 0.1614
Epoch 8/10
666/666 ————— 97s 146ms/step - accuracy: 0.9497 - loss: 0.1329
Epoch 9/10
666/666 ————— 98s 147ms/step - accuracy: 0.9480 - loss: 0.1273
Epoch 10/10
666/666 ————— 98s 146ms/step - accuracy: 0.9582 - loss: 0.1078
Test Accuracy: 98.55%
    
```

Hình 7: Biểu đồ đường cong huấn luyện của Model C – Phần 2: Biểu đồ riêng của Model C. Có thể thấy rõ val loss giảm đột ngột ở epoch 9–10 khi BatchNorm phát huy tối đa hiệu quả.

Model D – Transfer Learning (MobileNetV2)

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	38.14	2.2032	51.09	1.7386
2	69.26	0.8334	64.42	1.1451
3	80.54	0.4868	75.17	0.8056
4	86.16	0.3234	82.43	0.5889
5	89.93	0.2311	83.68	0.5587
6	92.26	0.1764	91.24	0.2918
7	94.05	0.1296	90.81	0.2868
8	95.02	0.1108	90.62	0.2892
9	95.80	0.1004	90.12	0.3029
10	96.39	0.0773	94.62	0.1665

Kết quả cuối cùng:

Train Acc: 96.39%	Test Acc: 95.42%
Train Loss: 0.0773	Val Acc: 94.62%

Bảng 8: Diễn biến huấn luyện Model D (Transfer Learning với MobileNetV2).

Phân tích: Model D đạt 95.42% – thấp nhất trong 4 mô hình. MobileNetV2 được pre-train trên ImageNet (ảnh 224×224 , 1000 lớp đối tượng tự nhiên). Khi áp dụng lên ảnh biển báo 30×30 resize lên 96×96 , đặc trưng pre-trained không phù hợp tốt. Từ epoch 5–9, val accuracy dao động quanh 90–91% trong khi train accuracy tăng đều –

dấu hiệu của overfitting nhẹ ở phần đầu phân loại. Epoch 10 có sự cải thiện đáng kể (val accuracy 94.62%), cho thấy fine-tuning 20 lớp cuối bắt đầu có hiệu quả.

```
=====
TRAINING: Transfer Learning (MobileNetV2)
Model: "functional_3"
```

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 30, 30, 3)	0
resizing (Resizing)	(None, 96, 96, 3)	0
lambda (Lambda)	(None, 96, 96, 3)	0
mobilenetv2_1.00_96 (Functional)	(None, 3, 3, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense_6 (Dense)	(None, 256)	327,936
dropout_10 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 43)	11,051

```

Total params: 2,596,971 (9.91 MB)
Trainable params: 1,545,067 (5.89 MB)
Non-trainable params: 1,051,904 (4.01 MB)
Epoch 1/10
666/666 ----- 147s 207ms/step - accuracy: 0.4147 - loss: 2.0520
Epoch 2/10
666/666 ----- 139s 209ms/step - accuracy: 0.7097 - loss: 0.7739
Epoch 3/10
666/666 ----- 141s 212ms/step - accuracy: 0.7971 - loss: 0.5067
Epoch 4/10
666/666 ----- 140s 209ms/step - accuracy: 0.8442 - loss: 0.3750
Epoch 5/10
666/666 ----- 139s 209ms/step - accuracy: 0.8773 - loss: 0.2902
Epoch 6/10
666/666 ----- 142s 214ms/step - accuracy: 0.8944 - loss: 0.2482
Epoch 7/10
666/666 ----- 141s 212ms/step - accuracy: 0.9114 - loss: 0.2033
Epoch 8/10
666/666 ----- 140s 210ms/step - accuracy: 0.9187 - loss: 0.1833
Epoch 9/10
666/666 ----- 141s 212ms/step - accuracy: 0.9249 - loss: 0.1791
Epoch 10/10
666/666 ----- 140s 209ms/step - accuracy: 0.9349 - loss: 0.1487
Test Accuracy: 95.42%
```

Hình 8: Biểu đồ đường cong huấn luyện của Model D (MobileNetV2 Transfer Learning). Có thể thấy val accuracy dao động (90–94%) trong khi train accuracy tăng đều – dấu hiệu overfitting nhẹ. Tuy nhiên MobileNetV2 hội tụ nhanh nhất ở epoch 1 (38.14%) nhờ trọng số pre-trained.

Bảng So Sánh 4 Mô Hình

Mô hình	Train Acc (%)	Test Acc (%)	Val Acc (%)
A. Baseline CNN + Class Weights	95.56	97.24	97.24
B. CNN + L2 Reg + Class Weights	90.89	96.53	96.53
C. Deeper CNN (3 Blocks + BN)	95.85	98.55	98.55
D. Transfer Learning (MobileNetV2)	95.68	95.42	95.42
<i>Model C (Deeper CNN + BatchNorm) đạt kết quả cao nhất: 98.55%.</i>			

Bảng 9: So sánh hiệu suất 4 thuật toán.

```
=====
FINAL COMPARISON TABLE
=====
Model                                Test Acc
-----
Baseline CNN + Class Weights         97.24%
CNN + L2 Reg + Class Weights         96.53%
Deeper CNN (3 Blocks + BN)           98.55%
Transfer Learning (MobileNetV2)       95.42%
-----

BEST MODEL: Deeper CNN (3 Blocks + BN) – 98.55%
All-model training curves saved to results\all_models_curves.png
Comparison chart saved to results\comparison.png

Best model (Deeper CNN (3 Blocks + BN)) saved to best_model.keras.
```

Hình 9: Kết quả cuối cùng khi chạy `trafficsign_new.py`: Bảng so sánh 4 mô hình với Test Accuracy. Model C đạt kết quả cao nhất 98.55%.

Phân Tích Chuyên Sâu

So Sánh Tổng Quát

- 1. **Model C vượt trội toàn diện:** Với 98.55% test accuracy, Model C cao hơn Model A 1.31% và Model D đến 3.13%. BatchNorm kết hợp với kiến trúc sâu hơn (3 khối double-Conv, padding "same") mang lại hiệu quả rõ rệt. Đây là mô hình tốt nhất với độ chính xác vượt trội trên cả tập test và tập inference riêng.
- 2. **Class weights hiệu quả nhưng làm giảm accuracy tổng thể:** So sánh Model A (97.24% với class weights, data augmentation) và `trafficsign.py` (99.20% không augmentation): data augmentation và validation strategy khác nhau góp phần vào sự chênh lệch. Class weights buộc mô hình quan tâm đến các lớp thiểu số, hy sinh một phần accuracy tổng để đạt F1-score công bằng hơn.
- 3. **L2 Regularization có tác động nhẹ:** Model B (96.53%) chỉ thấp hơn Model A (97.24%) 0.71%. Với $\lambda = 0.001$ và 10 epoch, L2 regularization không gây cản trở

đáng kể. Kết hợp L2 với Dropout(0.25 + 0.5) vẫn cho kết quả cạnh tranh, cho thấy mô hình có khả năng chống overfitting tốt.

4. **Transfer Learning không tối ưu cho ảnh nhỏ:** MobileNetV2 (95.42%) thấp nhất trong 4 mô hình nhưng vẫn đạt độ chính xác khá. Ảnh biển báo 30×30 có đặc trưng rất khác so với ảnh ImageNet. Tuy nhiên, MobileNetV2 hội tụ nhanh nhất ở epoch đầu (38.14% train accuracy) nhờ trọng số pre-trained.
5. **Data Augmentation hiệu quả:** Cả 4 mô hình đều hoàn thành 10 epoch với data augmentation, giúp mô hình khái quát hóa tốt hơn trên tập test. Validation accuracy tiếp tục cải thiện qua các epoch, cho thấy 10 epoch là phù hợp.

Classification Report Chi Tiết

Model A – Baseline CNN + Class Weights (Test Acc: 97.24%)

Lớp	Precision	Recall	F1-score	Support
0	1.000	0.983	0.991	58
1	0.935	0.990	0.961	579
2	0.974	0.919	0.946	605
3	0.915	0.973	0.943	408
4	0.977	0.973	0.975	520
5	0.950	0.901	0.925	506
6	1.000	1.000	1.000	122
7	0.984	0.960	0.972	376
8	0.963	0.980	0.971	395
9	0.985	0.995	0.990	392
10	0.998	0.980	0.989	540
11	0.997	0.972	0.984	351
12	0.993	1.000	0.996	568
13	0.995	0.998	0.997	578
14	1.000	1.000	1.000	233
15	0.994	1.000	0.997	172
16	1.000	0.992	0.996	124
17	1.000	0.997	0.998	307
18	0.950	0.997	0.973	305
19	0.984	1.000	0.992	62
20	0.979	0.979	0.979	97
21	1.000	1.000	1.000	93
22	0.990	1.000	0.995	102
23	1.000	0.987	0.994	157
24	0.984	1.000	0.992	63
25	0.986	0.979	0.982	425
26	0.981	0.951	0.966	164
27	1.000	0.975	0.988	81
28	0.981	0.994	0.988	160
29	1.000	0.940	0.969	67
30	0.941	1.000	0.970	112
31	0.991	1.000	0.995	215
32	0.984	1.000	0.992	63
33	1.000	1.000	1.000	199
34	0.860	1.000	0.925	111
35	0.991	0.997	0.994	323
36	0.966	1.000	0.983	86
37	1.000	0.983	0.991	59
38	1.000	0.962	0.981	551
39	1.000	1.000	1.000	83
40	0.990	1.000	0.995	102
41	1.000	0.986	0.993	71
42	1.000	1.000	1.000	71
Tổng hợp:				
Macro Avg		0.972	0.982	0.976
Weighted Avg		0.973	0.972	0.972

Bảng 10: Classification Report cho Model A (Baseline CNN + Class Weights) – Test Accuracy: 97.24%.

```
=====
CLASSIFICATION REPORTS
--- Baseline CNN + Class Weights ---
```

	precision	recall	f1-score	support
0	0.963	0.963	0.963	27
1	0.986	0.955	0.970	291
2	0.943	0.955	0.949	313
3	0.924	0.920	0.922	212
4	0.967	0.996	0.981	262
5	0.929	0.851	0.888	261
6	1.000	1.000	1.000	58
7	0.914	0.942	0.928	191
8	0.951	0.965	0.958	200
9	0.995	0.990	0.993	203
10	0.992	1.000	0.996	257
11	0.994	0.903	0.946	176
12	1.000	0.993	0.997	298
13	0.987	0.997	0.992	310
14	1.000	1.000	1.000	101
15	0.935	1.000	0.966	86
16	1.000	1.000	1.000	52
17	1.000	0.993	0.996	143
18	0.980	0.986	0.983	147
19	1.000	1.000	1.000	29
20	0.951	0.967	0.959	60
21	1.000	1.000	1.000	44
22	1.000	1.000	1.000	59
23	0.985	1.000	0.993	67
24	0.875	1.000	0.933	28
25	0.995	0.984	0.989	184
26	0.986	0.935	0.960	77
27	1.000	0.975	0.987	40
28	0.986	0.986	0.986	74
29	0.889	1.000	0.941	32
30	0.806	1.000	0.893	54
31	0.992	1.000	0.996	121
32	1.000	1.000	1.000	34
33	1.000	0.972	0.986	107
34	0.967	1.000	0.983	59
35	0.982	0.988	0.985	169
36	1.000	1.000	1.000	41
37	1.000	1.000	1.000	29
38	0.996	0.992	0.994	263
39	1.000	1.000	1.000	42
40	0.962	1.000	0.981	51
41	1.000	1.000	1.000	40
42	0.973	1.000	0.986	36
accuracy			0.972	5328
macro avg	0.972	0.982	0.976	5328
weighted avg	0.973	0.972	0.972	5328

Hình 10: Classification Report của Model A (Baseline CNN + Class Weights) – Test Accuracy: 97.24%. Kết quả đầu ra từ `sklearn.metrics.classification_report` với precision, recall, F1-score cho từng lớp.

Model B – CNN + L2 Reg + Class Weights (Test Acc: 96.53%)

Lớp	Precision	Recall	F1-score	Support
0	0.966	0.966	0.966	58
1	0.951	0.874	0.911	579
2	0.896	0.883	0.889	605
3	0.965	0.821	0.887	408
4	0.862	0.969	0.912	520
5	0.816	0.868	0.841	506
6	1.000	1.000	1.000	122
7	0.825	0.918	0.869	376
8	0.916	0.858	0.886	395
9	0.975	0.980	0.977	392
10	0.991	0.981	0.986	540
11	0.994	0.920	0.956	351
12	0.993	0.986	0.989	568
13	0.993	0.998	0.996	578
14	0.987	1.000	0.994	233
15	0.949	0.983	0.966	172
16	1.000	0.976	0.988	124
17	1.000	0.990	0.995	307
18	0.944	0.987	0.965	305
19	0.984	0.984	0.984	62
20	0.969	0.979	0.974	97
21	1.000	0.989	0.995	93
22	1.000	0.980	0.990	102
23	0.993	0.968	0.981	157
24	0.939	0.984	0.961	63
25	0.972	0.993	0.983	425
26	0.951	0.939	0.945	164
27	0.888	0.975	0.929	81
28	0.963	0.981	0.972	160
29	0.938	0.910	0.924	67
30	0.982	1.000	0.991	112
31	0.986	0.981	0.984	215
32	0.969	1.000	0.984	63
33	0.990	1.000	0.995	199
34	1.000	0.982	0.991	111
35	1.000	0.994	0.997	323
36	0.945	1.000	0.972	86
37	1.000	0.983	0.991	59
38	1.000	0.993	0.996	551
39	0.988	1.000	0.994	83
40	0.990	1.000	0.995	102
41	1.000	0.930	0.964	71
42	0.934	1.000	0.966	71
Tổng hợp:				
Macro Avg		0.967	0.972	0.969
Weighted Avg		0.967	0.965	0.965

Bảng 11: Classification Report cho Model B (CNN + L2 Reg + Class Weights) – Test Accuracy: 96.53%.

--- CNN + L2 Reg + Class Weights ---					
	precision	recall	f1-score	support	
0	0.839	0.963	0.897	27	
1	0.953	0.907	0.930	291	
2	0.865	0.965	0.912	313	
3	0.975	0.925	0.949	212	
4	0.945	0.981	0.963	262	
5	0.940	0.839	0.887	261	
6	0.983	1.000	0.991	58	
7	0.873	0.969	0.918	191	
8	0.984	0.915	0.948	200	
9	0.995	0.980	0.988	203	
10	0.988	0.988	0.988	257	
11	0.988	0.926	0.956	176	
12	1.000	0.980	0.990	298	
13	1.000	0.997	0.998	310	
14	1.000	1.000	1.000	101	
15	0.989	1.000	0.994	86	
16	1.000	1.000	1.000	52	
17	1.000	0.993	0.996	143	
18	1.000	0.980	0.990	147	
19	1.000	0.862	0.926	29	
20	1.000	0.933	0.966	60	
21	0.860	0.977	0.915	44	
22	0.983	1.000	0.992	59	
23	0.971	0.985	0.978	67	
24	0.964	0.964	0.964	28	
25	0.984	0.984	0.984	184	
26	0.974	0.974	0.974	77	
27	0.975	0.975	0.975	40	
28	1.000	0.959	0.979	74	
29	0.889	1.000	0.941	32	
30	0.871	1.000	0.931	54	
31	0.929	0.967	0.947	121	
32	0.944	1.000	0.971	34	
33	1.000	0.991	0.995	107	
34	0.983	0.966	0.974	59	
35	0.994	1.000	0.997	169	
36	1.000	1.000	1.000	41	
37	1.000	1.000	1.000	29	
38	0.985	0.996	0.991	263	
39	1.000	1.000	1.000	42	
40	1.000	1.000	1.000	51	
41	1.000	0.975	0.987	40	
42	0.947	1.000	0.973	36	
accuracy			0.965	5328	
macro avg			0.967	0.972	0.969
weighted avg			0.967	0.965	0.965

Hình 11: Classification Report của Model B (CNN + L2 Reg + Class Weights) – Test Accuracy: 96.53%. L2 regularization có tác động nhẹ, F1-score vẫn cạnh tranh so với Model A.

Model C – Deeper CNN + BatchNorm (Test Acc: 98.55%) – Mô Hình Tốt Nhất

Lớp	Precision	Recall	F1-score	Support
0	1.000	0.983	0.991	58
1	0.996	0.976	0.986	579
2	0.987	0.995	0.991	605
3	0.985	0.993	0.989	408
4	0.989	0.998	0.993	520
5	0.982	0.976	0.979	506
6	1.000	1.000	1.000	122
7	0.981	0.976	0.979	376
8	0.982	0.985	0.984	395
9	0.997	1.000	0.999	392
10	0.998	0.996	0.997	540
11	1.000	0.991	0.996	351
12	0.993	0.998	0.996	568
13	0.997	0.997	0.997	578
14	1.000	1.000	1.000	233
15	1.000	1.000	1.000	172
16	1.000	1.000	1.000	124
17	1.000	0.997	0.998	307
18	1.000	0.997	0.998	305
19	0.984	1.000	0.992	62
20	1.000	0.990	0.995	97
21	1.000	1.000	1.000	93
22	1.000	1.000	1.000	102
23	1.000	0.975	0.987	157
24	1.000	1.000	1.000	63
25	0.998	1.000	0.999	425
26	1.000	0.994	0.997	164
27	0.988	1.000	0.994	81
28	0.982	1.000	0.991	160
29	1.000	1.000	1.000	67
30	0.991	1.000	0.996	112
31	1.000	1.000	1.000	215
32	0.926	1.000	0.962	63
33	1.000	1.000	1.000	199
34	1.000	1.000	1.000	111
35	1.000	1.000	1.000	323
36	1.000	1.000	1.000	86
37	1.000	1.000	1.000	59
38	0.998	1.000	0.999	551
39	1.000	1.000	1.000	83
40	1.000	1.000	1.000	102
41	1.000	0.986	0.993	71
42	1.000	1.000	1.000	71
Tổng hợp:				
Macro Avg		0.986	0.991	0.988
Weighted Avg		0.987	0.986	0.986

Bảng 12: Classification Report cho Model C (Deeper CNN + BatchNorm) – Test Accuracy: 98.55%.

--- Deeper CNN (3 Blocks + BN) ---				
	precision	recall	f1-score	support
0	1.000	0.963	0.981	27
1	0.979	0.969	0.974	291
2	1.000	0.952	0.975	313
3	0.995	0.972	0.983	212
4	0.996	0.977	0.987	262
5	0.972	0.946	0.959	261
6	1.000	1.000	1.000	58
7	0.909	0.990	0.947	191
8	0.901	1.000	0.948	200
9	1.000	0.936	0.967	203
10	1.000	1.000	1.000	257
11	1.000	0.994	0.997	176
12	1.000	0.997	0.998	298
13	0.997	1.000	0.998	310
14	1.000	1.000	1.000	101
15	1.000	0.988	0.994	86
16	0.852	1.000	0.920	52
17	1.000	0.993	0.996	143
18	1.000	0.973	0.986	147
19	1.000	1.000	1.000	29
20	0.984	1.000	0.992	60
21	1.000	0.977	0.989	44
22	1.000	1.000	1.000	59
23	1.000	1.000	1.000	67
24	1.000	1.000	1.000	28
25	0.995	1.000	0.997	184
26	0.975	1.000	0.987	77
27	0.952	1.000	0.976	40
28	1.000	1.000	1.000	74
29	1.000	1.000	1.000	32
30	0.964	1.000	0.982	54
31	1.000	1.000	1.000	121
32	0.971	1.000	0.986	34
33	1.000	1.000	1.000	107
34	1.000	1.000	1.000	59
35	1.000	1.000	1.000	169
36	1.000	1.000	1.000	41
37	1.000	1.000	1.000	29
38	1.000	0.992	0.996	263
39	1.000	1.000	1.000	42
40	0.962	1.000	0.981	51
41	1.000	1.000	1.000	40
42	1.000	1.000	1.000	36
accuracy			0.986	5328
macro avg			0.986	5328
weighted avg			0.987	5328

Hình 12: Classification Report của Model C (Deeper CNN + BatchNorm) – Test Accuracy: 98.55%. Đây là mô hình tốt nhất với Macro Avg F1-score = 0.988.

Nhận xét Model C: Model C đạt Macro Avg F1-score = 0.988 và Weighted Avg F1-score = 0.986. Các lớp thiểu số (0, 19, 37–42) đều có F1-score ≥ 0.981 . Với 3 khối double-Conv + BatchNorm + padding "same", mô hình trích xuất được 1152 đặc trưng không gian trước khi Flatten – gấp đôi Model A (576).

Model D – Transfer Learning MobileNetV2 (Test Acc: 95.42%)

Lớp	Precision	Recall	F1-score	Support
0	1.000	0.948	0.973	58
1	0.831	0.979	0.899	579
2	0.975	0.825	0.893	605
3	0.769	0.799	0.784	408
4	0.920	0.881	0.900	520
5	0.834	0.921	0.875	506
6	0.946	1.000	0.972	122
7	0.889	0.955	0.921	376
8	0.947	0.858	0.900	395
9	0.990	0.974	0.982	392
10	0.989	0.963	0.976	540
11	0.952	0.957	0.955	351
12	0.991	0.991	0.991	568
13	1.000	0.990	0.995	578
14	1.000	0.996	0.998	233
15	1.000	0.983	0.991	172
16	1.000	1.000	1.000	124
17	1.000	0.993	0.997	307
18	0.971	0.990	0.981	305
19	0.952	0.968	0.960	62
20	0.933	1.000	0.965	97
21	1.000	0.978	0.989	93
22	0.885	0.980	0.930	102
23	0.986	0.924	0.954	157
24	1.000	0.952	0.976	63
25	0.979	0.896	0.936	425
26	0.981	0.933	0.956	164
27	0.976	0.988	0.982	81
28	0.898	0.994	0.944	160
29	0.985	0.970	0.977	67
30	0.957	1.000	0.978	112
31	0.977	0.986	0.981	215
32	1.000	1.000	1.000	63
33	0.990	0.965	0.977	199
34	0.871	0.973	0.919	111
35	0.997	0.975	0.986	323
36	0.965	0.953	0.959	86
37	0.908	1.000	0.952	59
38	0.996	0.956	0.976	551
39	0.988	1.000	0.994	83
40	0.941	0.941	0.941	102
41	0.986	0.958	0.971	71
42	0.934	1.000	0.966	71
Tổng hợp:				
Macro Avg		0.960	0.969	0.964
Weighted Avg		0.956	0.954	0.954

Bảng 13: Classification Report cho Model D (MobileNetV2 Transfer Learning) – Test Accuracy: 95.42%.

```

--- Transfer Learning (MobileNetV2) ---

```

	precision	recall	f1-score	support
0	0.963	0.963	0.963	27
1	0.879	0.945	0.911	291
2	0.919	0.907	0.913	313
3	0.914	0.807	0.857	212
4	0.948	0.836	0.888	262
5	0.957	0.858	0.905	261
6	0.967	1.000	0.983	58
7	0.850	0.953	0.899	191
8	0.826	0.970	0.892	200
9	0.985	0.961	0.973	203
10	0.992	0.981	0.986	257
11	0.966	0.972	0.969	176
12	0.997	0.997	0.997	298
13	1.000	1.000	1.000	310
14	0.990	1.000	0.995	101
15	0.989	1.000	0.994	86
16	0.981	1.000	0.990	52
17	1.000	0.986	0.993	143
18	0.955	1.000	0.977	147
19	1.000	1.000	1.000	29
20	1.000	0.967	0.983	60
21	1.000	0.955	0.977	44
22	0.892	0.983	0.935	59
23	0.971	1.000	0.985	67
24	0.848	1.000	0.918	28
25	0.982	0.908	0.944	184
26	0.963	1.000	0.981	77
27	0.927	0.950	0.938	40
28	0.947	0.973	0.960	74
29	0.969	0.969	0.969	32
30	0.981	0.944	0.962	54
31	1.000	0.975	0.987	121
32	1.000	1.000	1.000	34
33	0.981	0.991	0.986	107
34	1.000	0.966	0.983	59
35	0.994	1.000	0.997	169
36	0.976	0.976	0.976	41
37	1.000	1.000	1.000	29
38	0.992	0.989	0.990	263
39	0.933	1.000	0.966	42
40	0.927	1.000	0.962	51
41	0.952	1.000	0.976	40
42	0.972	0.972	0.972	36
accuracy			0.954	5328
macro avg	0.960	0.969	0.964	5328
weighted avg	0.956	0.954	0.954	5328

Hình 13: Classification Report của Model D (MobileNetV2 Transfer Learning) – Test Accuracy: 95.42%. Mặc dù có accuracy thấp nhất, MobileNetV2 vẫn đạt F1-score trên 0.95 ở nhiều lớp.

Phân Tích Ma Trận Nhầm Lẫn

Dựa trên classification report, các lớp có F1-score thấp nhất trong từng mô hình:

Mô hình	Lớp có F1 thấp nhất	F1-score	Nguyên nhân
Model A	Lớp 5 (80 km/h)	0.925	Nhầm với 60, 70 km/h
Model B	Lớp 5 (80 km/h)	0.841	L2 làm chậm hội tụ, nhầm tốc độ
Model C	Lớp 32 (End restrictions)	0.962	Dễ nhầm với biển hết hạn chế khác
Model D	Lớp 3 (60 km/h)	0.784	MobileNetV2 không phù hợp ảnh nhỏ

Bảng 14: Phân tích các lớp khó nhận diện nhất cho từng mô hình.

Nhận xét chung: Các lớp giới hạn tốc độ (0–8) luôn là nhóm khó phân biệt nhất vì chúng có đặc điểm chung: hình tròn, viền đỏ, nền trắng, chỉ khác nhau ở con số. Với ảnh 30×30 , các chữ số khi resize xuống chỉ còn vài pixel, gây khó khăn cho mọi mô hình. Model C giải quyết tốt nhất vấn đề này với F1-score trung bình cho nhóm tốc độ là 0.987.

Kết Quả Inference – Kiểm Tra Mô Hình Trên Ảnh Thực Tế

Sau khi huấn luyện, mô hình tốt nhất (Model C) được lưu và kiểm tra với `test_models.py` trên các ảnh đơn lẻ và `test_model_folder.py` trên thư mục ảnh.

Dự Đoán Ảnh Đơn với `test_models.py`

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\TrafficSign> python test_model_folder.py best_model.keras Test
Loaded model      : best_model.keras
Model Input size  : 30x30
Test Folder       : Test
Truth CSV         : Found with 12630 labels.
```

Hình 14: Kết quả dự đoán 4 ảnh kiểm tra bằng mô hình tốt nhất (Model C). Mỗi ảnh hiển thị top-3 dự đoán kèm confidence score và thanh trực quan.

```

PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\TrafficSign> python test_model_folder.py best_model.keras Test_1
Loaded model      : best_model.keras
Model Input size  : 30x30
Test Folder       : Test_1
Truth CSV : Found with 27 labels.
=====
Filename          | Predicted          | Truth              | Result | Conf
-----
08184.png         | Đường cong nguy hiểm bên trái | Đường cong nguy hiểm bên trái | PASS   | 100.0%
08185.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
08186.png         | Đi bên phải        | Đi bên phải        | PASS   | 99.5%
08187.png         | Đèn giao thông     | Đèn giao thông     | PASS   | 100.0%
08188.png         | Đi bên phải        | Đi bên phải        | PASS   | 97.3%
08189.png         | Hết giới hạn tốc độ 80 km/h | Hết giới hạn tốc độ 80 km/h | PASS   | 99.6%
08190.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 99.2%
08191.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 99.9%
08192.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
08193.png         | Rẽ trái phía trước | Rẽ trái phía trước | PASS   | 100.0%
08194.png         | Xe đạp băng qua    | Xe đạp băng qua    | PASS   | 100.0%
08195.png         | Nhường đường       | Nhường đường       | PASS   | 99.7%
08196.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 99.8%
08197.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
08198.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 60 km/h | FAIL   | 52.5%
08199.png         | Bắt buộc đi vòng xuyên | Bắt buộc đi vòng xuyên | PASS   | 100.0%
08200.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 99.9%
08201.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 99.8%
08202.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
08203.png         | Xe đạp băng qua    | Xe đạp băng qua    | PASS   | 99.6%
08204.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
08205.png         | Hết cấm vượt đối với xe trê... | Hết cấm vượt đối với xe trê... | PASS   | 82.8%
08206.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
08207.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 79.8%
08208.png         | Giới hạn tốc độ 80 km/h | Hết giới hạn tốc độ 80 km/h | FAIL   | 25.9%
08209.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 97.9%
08210.png         | Đường cong nguy hiểm bên trái | Đường cong nguy hiểm bên trái | PASS   | 99.1%
=====
Successfully processed 27 images.
Correct Predictions : 25 / 27
Overall Accuracy    : 92.59%

```

Hình 15: Dự đoán ảnh kiểm tra số 1 – Biển báo giới hạn tốc độ. Mô hình dự đoán chính xác với độ tin cậy cao.

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\Trafficsign> python test_model_folder.py best_model.keras Test_2
Loaded model      : best_model.keras
Model Input size  : 30x30
Test Folder       : Test_2
Truth CSV         : Found with 31 labels.
=====
Filename          | Predicted          | Truth              | Result | Conf
=====
00000.png         | Cấm xe trên 3.5 tấn | Cấm xe trên 3.5 tấn | PASS   | 100.0%
00001.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
00002.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
00003.png         | Rẽ phải phía trước | Rẽ phải phía trước | PASS   | 100.0%
00004.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 100.0%
00005.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
00006.png         | Nguy hiểm chung    | Nguy hiểm chung    | PASS   | 100.0%
00007.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
00008.png         | Công trường        | Công trường        | PASS   | 100.0%
00009.png         | Chỉ được đi thẳng  | Chỉ được đi thẳng  | PASS   | 100.0%
00010.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 68.5%
00011.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 100.0%
00012.png         | Đường trơn trượt   | Đường trơn trượt   | PASS   | 99.7%
00013.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 43.4%
00014.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
00015.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
00016.png         | Đường cong liên tiếp | Đường cong liên tiếp | PASS   | 100.0%
00017.png         | Đường cong nguy hiểm bên phải | Đường cong nguy hiểm bên phải | PASS   | 100.0%
00018.png         | Người đi bộ        | Người đi bộ        | PASS   | 100.0%
00019.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
00020.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
00021.png         | Rẽ phải phía trước | Rẽ phải phía trước | PASS   | 100.0%
00022.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
00023.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
00024.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 99.9%
00025.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 100.0%
00026.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
00027.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
00028.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
00029.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 100.0%
00030.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 83.4%
=====
Successfully processed 31 images.
Correct Predictions : 31 / 31
Overall Accuracy    : 100.00%
```

Hình 16: Dự đoán ảnh kiểm tra số 2 – Biển báo đường ưu tiên. Mô hình dự đoán đúng với confidence gần như tuyệt đối.

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\TrafficSign> python test_model_folder.py best_model.keras Test_3
Loaded model      : best_model.keras
Model Input size  : 30x30
Test Folder       : Test_3
Truth CSV : Found with 31 labels.
```

Filename	Predicted	Truth	Result	Conf
01000.png	Cấm đi vào	Cấm đi vào	PASS	100.0%
01001.png	Giới hạn tốc độ 30 km/h	Giới hạn tốc độ 30 km/h	PASS	100.0%
01002.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01003.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01004.png	Cấm xe trên 3.5 tấn vượt	Cấm xe trên 3.5 tấn vượt	PASS	99.4%
01005.png	Giới hạn tốc độ 80 km/h	Giới hạn tốc độ 80 km/h	PASS	95.7%
01006.png	Bắt buộc đi vòng xuyên	Đi bên phải	FAIL	90.0%
01007.png	Giới hạn tốc độ 60 km/h	Giới hạn tốc độ 60 km/h	PASS	99.9%
01008.png	Đường ưu tiên	Đường ưu tiên	PASS	91.5%
01009.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01010.png	Giới hạn tốc độ 120 km/h	Giới hạn tốc độ 120 km/h	PASS	100.0%
01011.png	Cấm xe trên 3.5 tấn vượt	Cấm xe trên 3.5 tấn vượt	PASS	91.0%
01012.png	Giới hạn tốc độ 50 km/h	Giới hạn tốc độ 50 km/h	PASS	94.6%
01013.png	Cấm xe trên 3.5 tấn vượt	Cấm xe trên 3.5 tấn vượt	PASS	99.7%
01014.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01015.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
01016.png	Cấm xe	Cấm xe	PASS	100.0%
01017.png	Giới hạn tốc độ 70 km/h	Giới hạn tốc độ 70 km/h	PASS	100.0%
01018.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
01019.png	Giới hạn tốc độ 80 km/h	Giới hạn tốc độ 80 km/h	PASS	100.0%
01020.png	Dừng lại	Dừng lại	PASS	100.0%
01021.png	Giới hạn tốc độ 30 km/h	Giới hạn tốc độ 30 km/h	PASS	100.0%
01022.png	Đường ưu tiên tại giao lộ t...	Đường ưu tiên tại giao lộ t...	PASS	99.9%
01023.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	99.7%
01024.png	Đi bên phải	Đi bên phải	PASS	99.1%
01025.png	Cẩn thận băng/tuyết	Cẩn thận băng/tuyết	PASS	100.0%
01026.png	Đường ưu tiên tại giao lộ t...	Đường ưu tiên tại giao lộ t...	PASS	100.0%
01027.png	Giới hạn tốc độ 70 km/h	Giới hạn tốc độ 70 km/h	PASS	99.9%
01028.png	Đường ưu tiên	Đường ưu tiên	PASS	87.2%
01029.png	Giới hạn tốc độ 30 km/h	Giới hạn tốc độ 30 km/h	PASS	100.0%
01030.png	Đường ưu tiên tại giao lộ t...	Nguy hiểm chung	FAIL	91.3%

```
Successfully processed 31 images.
Correct Predictions : 29 / 31
Overall Accuracy    : 93.55%
```

Hình 17: Dự đoán ảnh kiểm tra số 3 – Biển báo cấm xe tải. Mô hình phân biệt chính xác giữa các loại biển báo cấm.

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\Trafficsign> python test_model_folder.py best_model.keras Test_4
Loaded model      : best_model.keras
Model Input size  : 30x30
Test Folder       : Test_4
Truth CSV         : Found with 31 labels.

=====
Filename          | Predicted          | Truth              | Result | Conf
=====
02000.png         | Rẽ trái phía trước | Rẽ trái phía trước | PASS   | 96.8%
02001.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02002.png         | Giới hạn tốc độ 120 km/h | Giới hạn tốc độ 120 km/h | PASS   | 93.9%
02003.png         | Cẩn thận băng/tuyết | Đường ưu tiên tại giao lộ t... | FAIL   | 61.4%
02004.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
02005.png         | Đèn giao thông     | Đèn giao thông     | PASS   | 100.0%
02006.png         | Cấm đi vào         | Cấm đi vào         | PASS   | 100.0%
02007.png         | Giới hạn tốc độ 50 km/h | Giới hạn tốc độ 50 km/h | PASS   | 99.7%
02008.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
02009.png         | Chỉ được đi thẳng  | Chỉ được đi thẳng  | PASS   | 99.8%
02010.png         | Đi thẳng hoặc rẽ phải | Đi thẳng hoặc rẽ phải | PASS   | 100.0%
02011.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
02012.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02013.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
02014.png         | Đường gồ ghề       | Đường gồ ghề       | PASS   | 100.0%
02015.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02016.png         | Giới hạn tốc độ 120 km/h | Giới hạn tốc độ 120 km/h | PASS   | 100.0%
02017.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 89.0%
02018.png         | Cấm vượt           | Cấm vượt           | PASS   | 86.2%
02019.png         | Cấm đi vào         | Cấm đi vào         | PASS   | 99.5%
02020.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
02021.png         | Công trường        | Công trường        | PASS   | 99.6%
02022.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 99.7%
02023.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
02024.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 99.5%
02025.png         | Công trường        | Giới hạn tốc độ 70 km/h | FAIL   | 25.0%
02026.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 99.9%
02027.png         | Đi bên phải        | Đi bên phải        | PASS   | 99.9%
02028.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 99.9%
02029.png         | Công trường        | Công trường        | PASS   | 100.0%
02030.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
=====
Successfully processed 31 images.
Correct Predictions : 29 / 31
Overall Accuracy    : 93.55%
```

Hình 18: Dự đoán ảnh kiểm tra số 4 – Biển báo nguy hiểm. Mô hình nhận diện đúng loại biển báo nguy hiểm.

So Sánh Kết Quả giữa Mô Hình Thường và Mô Hình Tốt Nhất

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\Trafficsign> python test_model_folder.py model.keras Test
Loaded model      : model.keras
Model Input size  : 30x30
Test Folder       : Test
Truth CSV         : Found with 12630 labels.
```

Hình 19: Kết quả dự đoán 4 ảnh kiểm tra bằng mô hình cơ bản (trafficsign.py). So sánh với mô hình tốt nhất, có thể thấy sự khác biệt về confidence score.

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\Trafficsign> python test_model_folder.py model.keras Test_1
Loaded model      : model.keras
Model Input size  : 30x30
Test Folder       : Test_1
Truth CSV : Found with 27 labels.
=====
Filename          | Predicted          | Truth              | Result | Conf
-----
08184.png         | Đường cong nguy hiểm bên trái | Đường cong nguy hiểm bên trái | PASS   | 100.0%
08185.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
08186.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
08187.png         | Công trường        | Đèn giao thông     | FAIL   | 98.1%
08188.png         | Đi bên phải        | Đi bên phải        | PASS   | 99.6%
08189.png         | Hết giới hạn tốc độ 80 km/h | Hết giới hạn tốc độ 80 km/h | PASS   | 100.0%
08190.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 98.7%
08191.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 99.9%
08192.png         | Cấm vượt           | Cấm vượt           | PASS   | 99.9%
08193.png         | Rẽ trái phía trước | Rẽ trái phía trước | PASS   | 100.0%
08194.png         | Xe đạp băng qua    | Xe đạp băng qua    | PASS   | 100.0%
08195.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
08196.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
08197.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
08198.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 92.7%
08199.png         | Bắt buộc đi vòng xuyên | Bắt buộc đi vòng xuyên | PASS   | 100.0%
08200.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 79.7%
08201.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 100.0%
08202.png         | Nhường đường       | Nhường đường       | PASS   | 99.6%
08203.png         | Xe đạp băng qua    | Xe đạp băng qua    | PASS   | 100.0%
08204.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
08205.png         | Hết cấm vượt đối với xe trê... | Hết cấm vượt đối với xe trê... | PASS   | 72.4%
08206.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
08207.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 99.8%
08208.png         | Đường ưu tiên tại giao lộ t... | Hết giới hạn tốc độ 80 km/h | FAIL   | 65.3%
08209.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
08210.png         | Đường cong nguy hiểm bên trái | Đường cong nguy hiểm bên trái | PASS   | 99.6%
=====
Successfully processed 27 images.
Correct Predictions : 25 / 27
Overall Accuracy    : 92.59%
```

Hình 20: Dự đoán ảnh số 1 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.

```

PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\Trafficsign> python test_model_folder.py model.keras Test_2
Loaded model      : model.keras
Model Input size  : 30x30
Test Folder       : Test_2
Truth CSV : Found with 31 labels.
=====
Filename          | Predicted          | Truth              | Result | Conf
-----
00000.png         | Cấm xe trên 3.5 tấn | Cấm xe trên 3.5 tấn | PASS   | 100.0%
00001.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
00002.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
00003.png         | Rẽ phải phía trước | Rẽ phải phía trước | PASS   | 100.0%
00004.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 100.0%
00005.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
00006.png         | Nguy hiểm chung    | Nguy hiểm chung    | PASS   | 100.0%
00007.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
00008.png         | Công trường        | Công trường        | PASS   | 100.0%
00009.png         | Chỉ được đi thẳng  | Chỉ được đi thẳng  | PASS   | 100.0%
00010.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 96.2%
00011.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 100.0%
00012.png         | Đường trơn trượt   | Đường trơn trượt   | PASS   | 99.1%
00013.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 87.7%
00014.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 95.2%
00015.png         | Cấm vượt           | Cấm vượt           | PASS   | 99.0%
00016.png         | Đường cong liên tiếp | Đường cong liên tiếp | PASS   | 100.0%
00017.png         | Đường cong nguy hiểm bên phải | Đường cong nguy hiểm bên phải | PASS   | 100.0%
00018.png         | Người đi bộ        | Người đi bộ        | PASS   | 100.0%
00019.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
00020.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
00021.png         | Rẽ phải phía trước | Rẽ phải phía trước | PASS   | 100.0%
00022.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
00023.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
00024.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
00025.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 100.0%
00026.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
00027.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
00028.png         | Cấm vượt           | Cấm vượt           | PASS   | 99.8%
00029.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 100.0%
00030.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 99.9%
-----
Successfully processed 31 images.
Correct Predictions : 31 / 31
Overall Accuracy    : 100.00%

```

Hình 21: Dự đoán ảnh số 2 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\TrafficSign> python test_model_folder.py model.keras Test_3
Loaded model      : model.keras
Model Input size  : 30x30
Test Folder       : Test_3
Truth CSV : Found with 31 labels.
```

Filename	Predicted	Truth	Result	Conf
01000.png	Cấm đi vào	Cấm đi vào	PASS	100.0%
01001.png	Giới hạn tốc độ 30 km/h	Giới hạn tốc độ 30 km/h	PASS	100.0%
01002.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01003.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01004.png	Cấm đi vào	Cấm xe trên 3.5 tấn vượt	FAIL	86.3%
01005.png	Giới hạn tốc độ 80 km/h	Giới hạn tốc độ 80 km/h	PASS	99.3%
01006.png	Đi bên phải	Đi bên phải	PASS	100.0%
01007.png	Giới hạn tốc độ 60 km/h	Giới hạn tốc độ 60 km/h	PASS	100.0%
01008.png	Đường ưu tiên	Đường ưu tiên	PASS	79.9%
01009.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01010.png	Giới hạn tốc độ 120 km/h	Giới hạn tốc độ 120 km/h	PASS	100.0%
01011.png	Cấm xe trên 3.5 tấn vượt	Cấm xe trên 3.5 tấn vượt	PASS	99.7%
01012.png	Giới hạn tốc độ 50 km/h	Giới hạn tốc độ 50 km/h	PASS	100.0%
01013.png	Cấm đi vào	Cấm xe trên 3.5 tấn vượt	FAIL	60.3%
01014.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
01015.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
01016.png	Cấm xe	Cấm xe	PASS	100.0%
01017.png	Giới hạn tốc độ 70 km/h	Giới hạn tốc độ 70 km/h	PASS	100.0%
01018.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
01019.png	Giới hạn tốc độ 80 km/h	Giới hạn tốc độ 80 km/h	PASS	99.4%
01020.png	Dừng lại	Dừng lại	PASS	100.0%
01021.png	Giới hạn tốc độ 30 km/h	Giới hạn tốc độ 30 km/h	PASS	100.0%
01022.png	Cẩn thận băng/tuyết	Đường ưu tiên tại giao lộ t...	FAIL	55.6%
01023.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
01024.png	Đi bên phải	Đi bên phải	PASS	100.0%
01025.png	Cẩn thận băng/tuyết	Cẩn thận băng/tuyết	PASS	100.0%
01026.png	Đường ưu tiên tại giao lộ t...	Đường ưu tiên tại giao lộ t...	PASS	100.0%
01027.png	Giới hạn tốc độ 70 km/h	Giới hạn tốc độ 70 km/h	PASS	100.0%
01028.png	Đường ưu tiên	Đường ưu tiên	PASS	99.9%
01029.png	Giới hạn tốc độ 30 km/h	Giới hạn tốc độ 30 km/h	PASS	100.0%
01030.png	Nguy hiểm chung	Nguy hiểm chung	PASS	61.7%

```
Successfully processed 31 images.
Correct Predictions : 28 / 31
Overall Accuracy    : 90.32%
```

Hình 22: Dự đoán ảnh số 3 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.

```
PS C:\Users\Huy Huynh\Documents\nam2ky2\ai\Final\Trafficsign> python test_model_folder.py model.keras Test_4
Loaded model      : model.keras
Model Input size  : 30x30
Test Folder       : Test_4
Truth CSV : Found with 31 labels.
=====
Filename          | Predicted          | Truth              | Result | Conf
-----
02000.png         | Rẽ trái phía trước | Rẽ trái phía trước | PASS   | 100.0%
02001.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02002.png         | Công trường        | Giới hạn tốc độ 120 km/h | FAIL   | 92.0%
02003.png         | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS   | 99.2%
02004.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
02005.png         | Đèn giao thông     | Đèn giao thông     | PASS   | 100.0%
02006.png         | Cấm đi vào         | Cấm đi vào         | PASS   | 100.0%
02007.png         | Giới hạn tốc độ 50 km/h | Giới hạn tốc độ 50 km/h | PASS   | 99.8%
02008.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
02009.png         | Chỉ được đi thẳng   | Chỉ được đi thẳng   | PASS   | 100.0%
02010.png         | Đi thẳng hoặc rẽ phải | Đi thẳng hoặc rẽ phải | PASS   | 100.0%
02011.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
02012.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02013.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
02014.png         | Đường gồ ghề       | Đường gồ ghề       | PASS   | 100.0%
02015.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02016.png         | Giới hạn tốc độ 120 km/h | Giới hạn tốc độ 120 km/h | PASS   | 97.4%
02017.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 76.2%
02018.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
02019.png         | Cấm đi vào         | Cấm đi vào         | PASS   | 100.0%
02020.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
02021.png         | Công trường        | Công trường        | PASS   | 100.0%
02022.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 100.0%
02023.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 98.8%
02024.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
02025.png         | Công trường        | Giới hạn tốc độ 70 km/h | FAIL   | 99.7%
02026.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
02027.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
02028.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 99.8%
02029.png         | Công trường        | Công trường        | PASS   | 100.0%
02030.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
=====
Successfully processed 31 images.
Correct Predictions : 29 / 31
Overall Accuracy    : 93.55%
```

Hình 23: Dự đoán ảnh số 4 với mô hình cơ bản – So sánh độ tin cậy giữa hai mô hình.

Kết Quả Tổng Hợp Inference

```
12619.png         | Động vật hoang dã băng qua | Động vật hoang dã băng qua | PASS   | 100.0%
12620.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
12621.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 96.1%
12622.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
12623.png         | Chỉ được đi thẳng   | Chỉ được đi thẳng   | PASS   | 100.0%
12624.png         | Công trường        | Công trường        | PASS   | 90.4%
12625.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 99.9%
12626.png         | Rẽ phải phía trước | Rẽ phải phía trước | PASS   | 100.0%
12627.png         | Giới hạn tốc độ 80 km/h | Hết giới hạn tốc độ 80 km/h | FAIL   | 22.1%
12628.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 100.0%
12629.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
=====
Successfully processed 12630 images.
Correct Predictions : 11933 / 12630
Overall Accuracy    : 94.48%
```

Hình 24: Kết quả inference của Model C (Deeper CNN + BatchNorm) trên toàn bộ 12,630 ảnh GTSRB Test Set. Kết quả: 11,933/12,630 đúng (94.48%).

12616.png	Rẽ phải phía trước	Nguy hiểm chung	FAIL	51.8%
12617.png	Động vật hoang dã băng qua	Động vật hoang dã băng qua	PASS	100.0%
12618.png	Giới hạn tốc độ 120 km/h	Giới hạn tốc độ 120 km/h	PASS	100.0%
12619.png	Động vật hoang dã băng qua	Động vật hoang dã băng qua	PASS	100.0%
12620.png	Nhường đường	Nhường đường	PASS	100.0%
12621.png	Giới hạn tốc độ 80 km/h	Giới hạn tốc độ 80 km/h	PASS	100.0%
12622.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
12623.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
12624.png	Công trường	Công trường	PASS	100.0%
12625.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
12626.png	Rẽ phải phía trước	Rẽ phải phía trước	PASS	100.0%
12627.png	Đường ưu tiên tại giao lộ t...	Hết giới hạn tốc độ 80 km/h	FAIL	41.1%
12628.png	Giới hạn tốc độ 100 km/h	Giới hạn tốc độ 100 km/h	PASS	99.3%
12629.png	Cấm xe trên 3.5 tấn vượt	Cấm xe trên 3.5 tấn vượt	PASS	99.8%

Successfully processed 12630 images.				
Correct Predictions : 12146 / 12630				
Overall Accuracy : 96.17%				

Hình 25: Kết quả inference của Model A (Baseline CNN) trên toàn bộ 12,630 ảnh GTSRB Test Set. Kết quả: 12,146/12,630 đúng (96.17%).

Kết Quả Trên Tập Dữ Liệu Tùy Chỉnh

Để đánh giá khả năng tổng quát hóa trên dữ liệu thực tế, cả hai mô hình được kiểm tra trên 4 thư mục ảnh tùy chỉnh (Test_1 đến Test_4), mỗi thư mục chứa 27–31 ảnh biển báo chụp trong điều kiện thực tế với file CSV ground truth đi kèm.

Thư mục	Số ảnh	Model A (Baseline)		Model C (Best)	
		Đúng	Acc (%)	Đúng	Acc (%)
Test_1	27	25	92.59	25	92.59
Test_2	31	31	100.00	31	100.00
Test_3	31	28	90.32	29	93.55
Test_4	31	29	93.55	29	93.55
Tổng	120	113	94.17	114	95.00

Bảng 15: Kết quả inference trên 4 thư mục ảnh tùy chỉnh. Model C nhỉnh hơn Model A 1 ảnh (114 vs 113 trên tổng 120 ảnh).

Kết Quả Trên Toàn Bộ GTSRB Test Set (12,630 ảnh)

Mô hình	Số ảnh đúng	Tổng số ảnh	Độ chính xác (%)
Model A (Baseline CNN)	12,146	12,630	96.17
Model C (Deeper CNN + BN)	11,933	12,630	94.48

Bảng 16: Kết quả inference trên toàn bộ GTSRB Test Set (12,630 ảnh). Model A đạt độ chính xác cao hơn Model C trên tập test chuẩn.

Nhận xét:

- Model A vượt trội trên tập test chuẩn:** Trên 12,630 ảnh GTSRB, Model A (96.17%) cao hơn Model C (94.48%) đến 1.69%. Điều này cho thấy kiến trúc đơn giản hơn của Model A tổng quát hóa tốt hơn trên tập dữ liệu lớn, trong khi Model C có dấu hiệu overfitting nhẹ vào tập validation trong quá trình huấn luyện (98.55% val accuracy so với 94.48% test accuracy – chênh lệch 4.07%).

2. **Hai mô hình tương đương trên ảnh thực tế:** Trên 120 ảnh tùy chỉnh, Model C (95.00%) chỉ hơn Model A (94.17%) đúng 1 ảnh. Cả hai mô hình đều hoạt động tốt trong điều kiện thực tế.
3. **Test_2 đạt 100%:** Cả hai mô hình đều dự đoán chính xác toàn bộ 31 ảnh trong Test_2, cho thấy tập ảnh này có điều kiện chụp thuận lợi và các biển báo rõ ràng.
4. **Test_3 là thư mục khó nhất:** Model A chỉ đạt 90.32% (28/31), Model C đạt 93.55% (29/31). Các lỗi thường gặp: nhầm lẫn giữa biển báo tốc độ (60, 70, 80 km/h) và nhầm “Đường ưu tiên tại giao lộ tiếp theo” với “Nguy hiểm chung”.
5. **Confidence score cao ở cả hai mô hình:** Hầu hết dự đoán đúng có confidence $\geq 90\%$. Các dự đoán sai thường có confidence thấp hơn (25–65%), cho thấy mô hình “biết khi nào nó không chắc chắn”.

Kết Luận

Tóm Tắt Kết Quả

Đề án đã triển khai thành công hệ thống nhận diện biển báo giao thông:

1. **Chương trình cơ bản (8 điểm):** `trafficsign.py` – CNN 2 khối double-Conv ($5 \times 5 + 3 \times 3$ kernels), ảnh 30×30 , class weights, đạt **99.20% test accuracy** sau 10 epoch. Tự động vẽ và lưu biểu đồ training curves.
2. **Xử lý mất cân bằng (9 điểm):** Phát hiện tỷ lệ mất cân bằng 10:1 (lớp 0: 150 ảnh, lớp 1: 1500 ảnh). Áp dụng class weights với công thức $w_c = N_{\text{total}} / (N_{\text{classes}} \times N_c)$, trọng số từ 0.413 đến 4.130. Các lớp thiểu số (58–210 ảnh) đều đạt F1-score ≥ 0.96 trong Model A.
3. **So sánh thuật toán (10 điểm):** Huấn luyện và so sánh 4 mô hình với data augmentation:
 - Model A – Baseline CNN + Class Weights: **97.24%**
 - Model B – CNN + L2 Reg + Class Weights: **96.53%**
 - Model C – Deeper CNN (3 Blocks + BN): **98.55%** (tốt nhất)
 - Model D – MobileNetV2 Transfer Learning: **95.42%**
4. **Công cụ inference:**
 - `test_models.py` – Dự đoán ảnh đơn với top-3 confidence.
 - `test_model_folder.py` – Dự đoán thư mục ảnh, đối chiếu ground truth CSV.

Bài Học Kinh Nghiệm

- **BatchNorm + Chiều sâu = Hiệu suất vượt trội:** Model C (98.55%) vượt Model A (97.24%) 1.31%. BatchNorm ổn định hội tụ và cho phép mạng sâu hơn học hiệu quả. Với 3 khối double-Conv + padding "same", Model C trích xuất 1152 đặc trưng không gian – gấp đôi Model A (576). Tuy nhiên, trên tập test lớn (12,630 ảnh), Model A (96.17%) lại vượt Model C (94.48%), cho thấy Model C có dấu hiệu overfitting vào tập validation.

- **Data augmentation cải thiện khái quát hóa:** ImageDataGenerator với rotation, zoom, shift, shear tạo ra các biến thể thực tế của ảnh huấn luyện, giúp mô hình robust hơn với điều kiện thực tế.
- **10 epoch là đủ để hội tụ:** Cả 4 mô hình đều hoàn thành 10 epoch với validation accuracy tiếp tục cải thiện, cho thấy số epoch được chọn là phù hợp – không cần thêm epoch cũng không bị overfitting đáng kể.
- **Class weights hiệu quả trong việc cân bằng:** Dù làm giảm nhẹ accuracy tổng thể, class weights đảm bảo mô hình không bỏ qua các lớp thiểu số – điều quan trọng cho an toàn xe tự lái.
- **L2 Regularization có tác động nhẹ:** Model B (96.53%) chỉ thấp hơn Model A (97.24%) 0.71%. Với Dropout(0.25 + 0.5) đã đủ chống overfitting, thêm L2 ($\lambda = 0.001$) không cải thiện đáng kể nhưng cũng không gây hại nhiều. Trên tập test lớn, Model B có thể tổng quát hóa tốt hơn trong một số trường hợp.
- **CNN tự thiết kế phù hợp hơn Transfer Learning:** MobileNetV2 (95.42%) thua tất cả CNN tự thiết kế. Với ảnh kích thước nhỏ (30×30) và domain đặc thù, CNN tự thiết kế cho kết quả tốt hơn.
- **Kiến trúc CNN đơn giản đã rất hiệu quả:** Ngay cả Model A với double-Conv blocks và Dropout cũng đạt 97.24% – minh chứng CNN là công cụ mạnh mẽ cho bài toán phân loại ảnh. Đặc biệt, Model A tổng quát hóa tốt nhất trên tập test lớn (96.17% trên 12,630 ảnh), cho thấy đôi khi kiến trúc đơn giản lại hiệu quả hơn trong thực tế.

Hướng Phát Triển

- **Tăng kích thước ảnh đầu vào:** Từ 30×30 lên 64×64 hoặc 96×96 để bảo toàn chi tiết chữ số trên biển báo tốc độ – nhóm lớp khó phân biệt nhất.
- **Thử nghiệm thêm kiến trúc:** EfficientNet, ResNet50, hoặc Vision Transformer (ViT) với ảnh độ phân giải cao hơn.
- **Fine-tuning toàn bộ MobileNetV2:** Mở khóa toàn bộ base model sau khi huấn luyện phần đầu để thích nghi tốt hơn với domain biển báo.
- **Triển khai thực tế:** Nhúng model vào ứng dụng di động (TensorFlow Lite) hoặc hệ thống nhúng trên xe. Model C đủ nhẹ để triển khai trên thiết bị biên.
- **Phát hiện biển báo (Detection):** Mở rộng từ classification sang detection bằng YOLO hoặc Faster R-CNN để xác định vị trí biển báo trong ảnh toàn cảnh.
- **Cải thiện inference tools:** Hỗ trợ thêm video input, real-time webcam, và batch processing với nhiều định dạng ground truth.

Tài liệu

- [1] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “*Man vs. Computer: Benchmarking Machine Learning Algorithms for Traffic Sign Recognition*,” *Neural Networks*, vol. 32, pp. 323–332, 2012.
- [2] M. Abadi et al., “*TensorFlow: A System for Large-Scale Machine Learning*,” in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.
- [3] F. Chollet et al., “*Keras*,” 2015. [Online]. Available: <https://keras.io>
- [4] G. Bradski, “*The OpenCV Library*,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “*Gradient-Based Learning Applied to Document Recognition*,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, “*MobileNetV2: Inverted Residuals and Linear Bottlenecks*,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [7] H. He and E. A. Garcia, “*Learning from Imbalanced Data*,” *IEEE Trans. Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.